

Arithmetic Expression Geometry IV: Projective Condensation and Computational Complexity

From Histories and Quotients to Representation and Cost

Mingli Yuan
Swarma Research
Beijing 100083, China

October 9, 2026

Draft manuscript under mathematical review

Draft notice. This manuscript is a working draft in the editorial and publication sense. Its conceptual organization, exposition, figures, and text–figure integration remain under development; the archived legacy manuscript may currently provide a more polished visual and expository experience. Here “draft” is not a mathematical claim-status label in the repository governance system and does not alter the stated status of any theorem, proposition, computation, conjecture, or open problem.

Abstract

Arithmetic Expression Geometry retains the ordered evaluation history of an expression before passing to its induced operator or endpoint. Papers I–III develop the affine, analytic, and singular-zero sectors of this programme. We study here the quotient architecture itself and ask when information forgotten by a quotient can be related to representation or computational resources.

For non-degenerate bilateral arithmetic over a field K , projective evaluation lands in $G = \mathrm{PGL}_2(K)$. A regular bivaluation is an ordered pair of distinct points of $\mathbb{P}^1(K)$ and is equivalently a rank-one idempotent on K^2 . If H stabilizes a reference ordered pair and $B_\pm$ stabilize its two members, then

$$\mathrm{Hist}_K \longrightarrow G \longrightarrow G/H \longrightarrow G/B_\pm$$

separates literal histories, projective operators, image–kernel projectors, and single projective points. The fibers have different meanings: the first is a relation fiber of words, while the second is an H -torsor of projective frames. Over $\mathbb{F}_q$ their sizes are completely explicit.

The central result identifies the exact condition under which a quotient state is sufficient for future continuation. Left continuation acts canonically on G/H , and, with identity continuation and an injective observation, its contextual residual classes are exactly the points of G/H . A single right update by k descends exactly when $k^{-1}Hk \subseteq H$, and reversible right continuation descends exactly through the normalizer $N_G(H)$. Thus a quotient can be a complete online state for one chronology while the same discarded stabilizer direction becomes observable for the opposite chronology. This gives a precise bridge from projective condensation to future-sensitive information; quotient cardinality alone is not such a bridge.

We then define contextual continuation residuals for a typed interface, a class of legal futures, and a declared observation, including a distinguished undefined outcome for partial arithmetic. In a finite deterministic exact model the residual quotient is the canonical minimal online state

space. Any lossless implementation therefore needs at least $\lceil \log_2 |\mathcal{R}_C| \rceil$ bits at the cut C . Under a declared fixed-register snapshot convention, the same argument gives a corresponding capacity–time lower bound. This semantic bound is kept separate from an operational trace of live configurations, whose actions may compute, erase, communicate, checkpoint, or recompute. A monotone set of nodes ever computed cannot determine workspace once erasure and recomputation are allowed.

For a ported AES we distinguish geometric punctures, computational holes, and ordered input/output ports. Finite typed acyclic networks with explicit copy and strict discard admit source- and occurrence-preserving substitution. Substitution commutes with partial evaluation on exactly the composite domain; network associativity holds up to an explicit isomorphism while literal call receipts remain distinct. Compatible AES collars and computational interfaces descend together under a global acyclicity hypothesis. Feature observation sufficiency is equivalent to refinement of contextual equivalence; online feature updates additionally require a right congruence. A four-input, three-output rational example illustrates this interface without claiming that its attached gates have been realized as AES motions.

Several exact calibrations show both the reach and the limits of the framework. Binary Horner histories with fixed length and Hamming weight have one ACS charge but $\binom{n}{m}$ distinct affine operators. For a finite alphabet, a simple fiber–entropy inequality relates raw word growth to evaluation-image growth, without turning either into a runtime lower bound. The equality function has OBDD width at least 2^n under a block order but constant width under an interleaved order, although variable restrictions commute. A radix-2 butterfly factors as a common matrix times a diagonal torus label; a full NTT still requires a multi-wire linear category, central-scalar bookkeeping, and a cost model distinguishing static twiddles from dynamic states. Matrix-chain parenthesizations and reverse-mode checkpointing exhibit genuine work–space tradeoffs among equal semantic realizations. Potential-difference labels and pure-gauge group labels telescope on every rewrite loop, so they cannot by themselves define a nontrivial process holonomy.

The resulting theory has four layers: projective quotient fibers, contextual residuals, operational live configurations, and rewrite fibers. We prove comparison theorems among them only when the required actions and observations are specified. We do not infer computational hardness from noncommutativity, negative curvature, exponential group growth, or the size of a raw history fiber. The remaining programme is to construct non-flat projective transport, faithful multi-wire AES motion realizations, actual opposite-edge feature witnesses, costed AEG-native representation changes, and approximate residual theories.

Keywords: arithmetic expression geometry; projective linear group; homogeneous space; rank-one idempotent; quotient information; residual equivalence; online state; word growth; decision diagram; FFT; checkpointing; time–space tradeoff

Contents

1	Introduction	6
1.1	Four layers	6
1.2	Main results	6
1.3	What is not claimed	8
1.4	Roadmap	8
2	Histories, evaluations, and equality levels	8
2.1	Free histories and two semantic categories	8
2.2	The equality tower	9
2.3	Group-valued relative defects	10
2.4	Relations, neutral histories, and fibers	10
2.5	Chronological end versus operand slot	11

3	Bivaluations and the projective quotient tower	11
3.1	Points, copoints, and regular bivaluations	11
3.2	The homogeneous-space tower	12
3.3	A third point restores the projective frame	12
3.4	Finite-field calibration	13
3.5	Algebraic and dynamical condensation	14
4	Condensation and future continuation	14
4.1	Equivariant quotients and future sufficiency	14
4.2	Left continuation on the bivaluation quotient	15
4.3	Right continuation and the normalizer boundary	15
4.4	The split-torus normalizer in rank one	16
4.5	Sufficient, exact, and minimal condensations	16
4.6	An exact-transport obstruction	16
5	Contextual residuals and minimal exact state	17
5.1	Typed interfaces and totalized partial action	17
5.2	Residuals form the canonical continuation machine	18
5.3	Finite-state information lower bound	19
5.4	Three projective residual experiments	19
5.5	Randomized and approximate boundaries	20
6	Ported AES: substitution, execution, and composition	20
6.1	Three kinds of opening	20
6.2	Finite programs, sources, and occurrences	20
6.3	Program substitution and its receipts	21
6.4	Substitution and partial evaluation	22
6.5	Joint geometric and computational descent	23
6.6	Features and future computation	23
6.7	A four-puncture, four-input, three-output experiment	25
6.8	Finite checks and remaining obligations	26
7	Operational resource geometry	26
7.1	Logical cuts and live configurations	26
7.2	Why completed-node sets do not determine space	27
7.3	Embedding the residual lower bound	27
7.4	Uniform families and Pareto resource geometry	28
7.5	Three distinct order effects	28
8	Word growth, fibers, and information loss	28
8.1	Words, image growth, and fiber multiplicity	29
8.2	A conditional-information form	29
8.3	Word metrics and coarse growth	29
8.4	Four growth functions that must not be merged	30
8.5	Geometry requires a comparison map	30

9	Rewrite fibers, normal forms, and exactness obstructions	30
9.1	Rewrite graphs and normal forms	31
9.2	Potential labels are exact	31
9.3	Action is not holonomy	32
9.4	Large fibers can have shared representations	32
9.5	The non-exact transport problem	32
10	Horner histories and ACS condensation	32
10.1	Radix histories	33
10.2	Condensation by ACS charge	33
10.3	Which state is sufficient?	34
10.4	Streaming cost and encoding cost	34
11	Cut order and residual width in decision diagrams	34
11.1	Residual functions after a variable prefix	34
11.2	Equality under block order	35
11.3	Equality under interleaved order	35
11.4	The restrictions commute	35
11.5	What the width theorem measures	35
12	Butterflies, transform representations, and NTT states	36
12.1	The butterfly torus-coset lemma	36
12.2	Why linear-group data remain necessary	36
12.3	Transform factorization of convolution	36
12.4	A finite NTT calibration	37
12.5	Static labels versus dynamic states	37
13	Equal semantics with different work and space	38
13.1	Matrix-chain parenthesization	38
13.2	A chain and its reverse sweep	38
13.3	History condensation and re-expansion	39
13.4	Reversible computation boundary	39
13.5	What equal semantics permits us to conclude	39
14	Claim boundaries and the remaining programme	39
14.1	The four layers	40
14.2	Principal proved statements	40
14.3	Excluded implications	41
14.4	Interfaces with Papers 0–III	41
14.5	Open problems	42
15	Conclusion	42
A	Projective and homogeneous-space calculations	43
A.1	One-hole arithmetic contexts	43
A.2	Generation of the projective group	44
A.3	Point–predicate covariance	45
A.4	Local affine bivaluation law	45
A.5	Frame coordinate above a bivaluation	45

A.6	Direct normalizer calculation	46
A.7	Finite-field count from ordered frames	46
B	Residual proofs and partial-domain conventions	46
B.1	Why an invalid outcome is necessary	46
B.2	The factorization diagram	46
B.3	One-way communication interpretation	47
B.4	Dense tensor boundary example	47
B.5	Prefix-free variable-length states	47
B.6	Infinite residual spaces	47
B.7	Changing future classes	48
C	Finite network and resource counts	48
C.1	The finite-field projective tower	48
C.2	Binary Horner words of length four	48
C.3	Free-group ball count	49
C.4	Finite-field roots and transform networks	49
C.5	Matrix-chain calibration	49
C.6	Reverse-chain endpoint schedules	49
D	Claim ledger and cost-model checklist	50
D.1	Claim ledger	50
D.2	Cost-model checklist	52
D.3	Case-study matrix	53
D.4	Release status	53

1 Introduction

Arithmetic evaluation is ordinarily described by a map from an expression to its value. Arithmetic Expression Geometry begins one level earlier. It treats an ordered evaluation history as mathematical data and asks what survives when that history is replaced successively by an operator, a quotient state, or an endpoint. Paper I supplies the marked-history formalism and projective evaluation [11], and Paper 0 supplies the affine sector, the ACS charge shadow, and the contact curvature [1]. Paper II develops analysis on the regular affine spaces [12], and Paper III studies singular zero geometry and topological transport [13]. The present paper returns to the forgetful maps themselves.

The motivating hierarchy is

$$\text{marked history} \longrightarrow \text{projective operator} \longrightarrow \text{quotient state} \longrightarrow \text{endpoint or observation.} \quad (1)$$

Every arrow can identify distinct inputs. It is tempting to call the resulting fiber “lost information” and to identify its logarithmic size with memory, representation complexity, or reconstruction time. Such an identification is not valid without further data. A fiber is an algebraic or set-theoretic object. Computational complexity belongs to an encoded family of tasks in a specified machine model. The missing bridge is a theorem saying which distinctions in a fiber can still affect an allowed future.

1.1 Four layers

We organize the subject into four layers.

1. **Projective quotient fibers.** Bilateral arithmetic histories evaluate in $G = \text{PGL}_2(K)$. Homogeneous quotients such as G/H forget a projective frame direction while retaining an image–kernel projector.
2. **Contextual residuals.** At a typed cut C , two pasts are equivalent when every declared future gives the same observation, including the distinguished illegal outcome. This equivalence depends on the future class, the observation, and the treatment of undefined arithmetic.
3. **Operational live configurations.** A run stores concrete values, checkpoints, messages, or clauses. Its space is the encoded size of what is live, not the size of an abstract quotient fiber and not the set of nodes that have ever been computed.
4. **Rewrite fibers.** Equal semantic objects may have many factorizations, parenthesizations, or schedules. Rewrite distance, normal-form cost, filling area, and possible holonomy are properties of this realization fiber, not coordinates of an endpoint alone.

Figure 1 records the direction of comparison used in this paper. The vertical arrows are not asserted to be isomorphisms.

1.2 Main results

The paper establishes seven groups of results.

First, we complete the algebraic quotient layer. A regular bivaluation—an ordered pair of distinct projective points—is naturally a rank-one idempotent. For the ordered-pair stabilizer H and the two point stabilizers $B_{\pm}$,

$$G/H \cong \text{Biv}_K^{\text{reg}}, \quad G/B_{\pm} \cong \mathbb{P}^1(K).$$

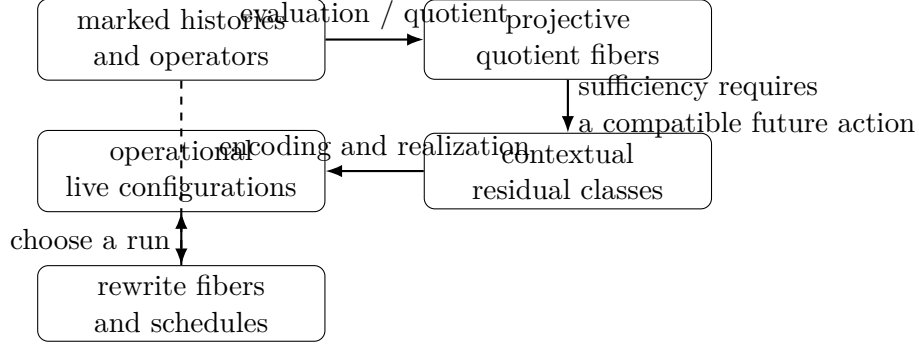


Figure 1: The four layers of Paper IV. A projective quotient becomes a future-sensitive state only after the future action and observation are fixed; a residual lower bound becomes operational space only after encoding and machine conventions are fixed.

The principal H -torsor $G \rightarrow G/H$ is the projective-frame residue over a condensed projector. This is algebraic condensation. It must be distinguished from convergence of normalized matrix products to a singular rank-one map.

Second, we prove a continuation-side theorem. Left multiplication by every future operator descends to G/H . A single right update by k descends exactly when $k^{-1}Hk \subseteq H$; a reversible right action descends exactly when k normalizes H . Consequently, the same quotient may be sufficient for chronological continuation by left composition and insufficient for a right-composition convention. The theorem gives an exact meaning to the statement that forgotten frame data can become visible again.

Third, we define contextual continuation residuals and prove the finite exact minimal-state theorem. The quotient by residual equivalence is a canonical deterministic online state space, in the same structural tradition as the Nerode equivalence for automata [7]. Any exact implementation must inject this quotient into its encoded states. This yields a bit lower bound and, under an explicit fixed-register snapshot convention, a capacity–time lower bound.

Fourth, we construct a finite ported-program interface for punctured AES carriers. Source- and occurrence-preserving substitution is compatible with strict partial evaluation on its exact domain. Network reassociation is an explicit isomorphism, not equality of literal call receipts. Compatible geometric collars and computational interfaces descend together when the assembled event graph is acyclic. Feature observation factorization and online feature update have distinct criteria: contextual refinement and a right-congruence kernel, respectively. The attached-program construction does not supply a faithful realization of its gates by AES motions.

Fifth, we introduce a live-configuration cost vector separating static description, work, elapsed steps, causal depth, peak workspace, memory–time, and communication. We prove that a monotone completed-node set cannot encode space in models allowing deletion or recomputation.

Sixth, we give a word-fiber and conditional-entropy inequality. Exponential raw history growth combined with small evaluation-image growth forces large operator fibers and large conditional information. The conclusion is about an evaluation map. It is not a time lower bound. Finite generating sets give quasi-isometric word metrics, so polynomial versus exponential group growth is stable under that restricted change of generators, while computational costs remain model-dependent.

Seventh, exact case studies calibrate the distinctions. Horner words exhibit charge condensation; OBDDs exhibit future-residual sensitivity to variable order; butterflies and NTTs exhibit representation change and static torus labels; matrix chains and checkpointing exhibit equal semantics

with different work and space; and rewrite potentials exhibit a telescoping obstruction to nontrivial holonomy.

1.3 What is not claimed

No result in this paper asserts any of the implications

$$\begin{aligned} \text{noncommutativity} &\implies \text{negative curvature}, & \text{negative curvature} &\implies \text{exponential time}, \\ \text{exponential word growth} &\implies \text{NP-hardness}, & \log |\text{fiber}| &= \text{workspace}. \end{aligned}$$

The free group already has exponential metric growth while free reduction is linear in the input-word length. Conversely, the OBDD example in Section 11 has an exponential cut-width separation although the underlying variable restrictions commute. The appropriate conclusions are therefore conditional comparison theorems, not a universal scalar complexity law.

1.4 Roadmap

Sections 2–3 establish the history and homogeneous-space layers. Section 4 proves the quotient-continuation results. Section 5 develops residual semantics. Section 6 supplies the finite ported-program interface, and Section 7 separates semantic state from operational resources. Sections 8 and 9 treat growth and rewriting. The four case-study sections follow. The final two sections state the claim boundary and the remaining research programme. Appendices collect calculations, proof variants, finite counts, and a theorem-level claim ledger.

2 Histories, evaluations, and equality levels

Paper I defines a marked spinal history as a chronological word of one-hole arithmetic contexts. We recall only the structure needed to discuss quotient fibers and continuation. Throughout this section K is a field, and all projective contexts are assumed non-degenerate.

2.1 Free histories and two semantic categories

Let Σ_K be a selected set of invertible one-hole arithmetic contexts. It may contain translations, nonzero scalings, sign changes, and inversion. A free history is a word

$$\gamma = s_1 s_2 \cdots s_n, \quad s_i \in \Sigma_K,$$

listed in chronological order. Its induced transformation is

$$\rho(\gamma) = \rho(s_n) \cdots \rho(s_2) \rho(s_1) \in G := \text{PGL}_2(K). \quad (2)$$

Thus a newly appended future context acts by left multiplication. This chronology convention is essential in Section 4.

Definition 2.1 (Ordinary and projective evaluation). For an initial value $x \in K$, ordinary evaluation $\nu_x(\gamma)$ is defined only when every intermediate arithmetic operation is defined in K . Projective evaluation is the everywhere-defined action of $\rho(\gamma)$ on $\mathbb{P}^1(K)$. We write

$$\text{dom}_{\text{ord}}(\gamma) \subseteq K$$

for the ordinary domain.

For example, the projective continuation $z \mapsto c/z$ sends 0 to ∞ , while ordinary division by zero is undefined. The projective target therefore does not erase the ordinary-domain label. Later residual definitions will assign a distinguished observation $\perp$ to an illegal ordinary continuation.

Definition 2.2 (Word-labelled history category). Let $\mathcal{H}_K$ have objects $q \in \mathbb{P}^1(K)$. A morphism from q to q' is a pair (γ, q) consisting of a literal context word with $\rho(\gamma)q = q'$. If γ is followed chronologically by δ , the composite is labelled by the concatenated history $\gamma \star \delta$ and satisfies

$$\rho(\gamma \star \delta) = \rho(\delta)\rho(\gamma).$$

Formal inverses of context symbols give a groupoid completion $\mathcal{H}_K^{\text{inv}}$. Projective evaluation defines a functor

$$\rho : \mathcal{H}_K^{\text{inv}} \longrightarrow \mathbb{P}^1(K) \parallel G,$$

to the action groupoid by forgetting the literal word and retaining its projective operator. Distinct word-labelled arrows are not identified merely because they have the same image in G .

This construction is intentionally modest. Associativity, commutativity, distributivity, and program transformations may later be added as rewrite 2-cells between word-labelled arrows. They are not imposed as literal history equality.

2.2 The equality tower

A bounded history (x, γ) supports several non-equivalent equality tests. For compatible histories γ and δ , consider:

$\gamma = \delta$	literal history equality,
$\rho(\gamma) = \rho(\delta)$	projective operator equality,
$\rho(\gamma)q = \rho(\delta)q$	equality at a projective state q ,
$\nu_x(\gamma) = \nu_x(\delta)$	ordinary endpoint equality,
$\chi(\gamma) = \chi(\delta)$	equality of a selected charge shadow.

Only the forward implications forced by the selected maps are automatic. Endpoint equality at one input does not imply operator equality; operator equality does not imply history equality; and charge equality need not imply operator equality. These failures are not pathologies. They are the fibers that the present paper studies.

Proposition 2.3 (Nested kernel pairs). *Let $X \xrightarrow{f} Y \xrightarrow{g} Z$ be maps. The kernel pair of f is contained in the kernel pair of $g \circ f$:*

$$f(x) = f(x') \implies g(f(x)) = g(f(x')).$$

The converse holds exactly when g is injective on $f(X)$.

Proof. The forward implication is functoriality of equality. If g is injective on $f(X)$, equality after g implies equality before g . Conversely, if the converse implication holds for all x, x' , then g is injective on the image of f . $\square$

This proposition is the set-theoretic skeleton of the full tower. Its computational interpretation requires a future class, developed in Section 5.

2.3 Group-valued relative defects

Endpoint subtraction is natural in an affine chart but not projectively invariant. The operator group supplies a comparison that survives inversion.

Definition 2.4 (Relative projective defect). For invertible histories γ and δ , define

$$\mathcal{K}(\gamma, \delta) := \rho(\delta)^{-1} \rho(\gamma) \in G. \quad (3)$$

It satisfies

$$\mathcal{K}(\gamma, \gamma) = 1, \quad \mathcal{K}(\gamma, \delta)^{-1} = \mathcal{K}(\delta, \gamma),$$

and the cocycle identity

$$\mathcal{K}(\gamma, \eta) = \mathcal{K}(\delta, \eta) \mathcal{K}(\gamma, \delta). \quad (4)$$

Proposition 2.5 (Affine torsion as a coordinate of the relative defect). *Suppose*

$$\rho(\gamma)(x) = sx + A_\gamma, \quad \rho(\delta)(x) = sx + A_\delta, \quad s \neq 0.$$

Then

$$\mathcal{K}(\gamma, \delta)(x) = x + \frac{A_\gamma - A_\delta}{s}.$$

The ordinary endpoint defect $A_\gamma - A_\delta$ is therefore the translation coordinate of the projective relative defect, multiplied by the common scale.

Proof. The inverse of $x \mapsto sx + A_\delta$ is $x \mapsto s^{-1}(x - A_\delta)$. Composition with the first affine map gives the formula. $\square$

Once the two operators do not have the same affine linear part, a scalar endpoint difference depends on the initial value. Once inversion is admitted, it may also cross poles. The group-valued defect remains defined at the operator level, while its endpoint observation still requires domain data.

2.4 Relations, neutral histories, and fibers

Definition 2.6 (Operator relation and neutrality). An ordered pair (γ, δ) is an operator relation when $\rho(\gamma) = \rho(\delta)$. A history γ is operator-neutral when $\rho(\gamma) = 1$. It is charge-neutral relative to a charge map χ when $\chi(\gamma)$ equals the charge of the empty history.

Operator-neutral histories form the kernel of ρ only when the history language has been given a compatible group structure. In the free monoid they form a submonoid preimage of the identity; in the formal groupoid they form isotropy arrows above the relevant object. Charge-neutral and operator-neutral histories need not coincide.

Example 2.7 (Four elementary separations). The following examples use affine maps and are proved by direct composition.

1. $x \mapsto x + 1$ and $x \mapsto 2x$ agree at $x = 1$ but are different operators.
2. The words “add 1, then add 2” and “add 2, then add 1” are distinct histories inducing the same map $x \mapsto x + 3$.
3. “add p , then scale by e^q ” and the opposite order have the same ACS endpoint but differ by translation $p(e^q - 1)$.

4. “add p , then add $-p$ ” is a nonempty operator-neutral history.

The operator fiber $\rho^{-1}(g)$ and the endpoint fiber $\{\gamma : \nu_x(\gamma) = y\}$ are therefore distinct objects. The first is independent of x ; the second need not be. Paper IV uses “history fiber” only after naming the map whose fiber is intended.

2.5 Chronological end versus operand slot

The side on which a future operator is multiplied is not the operand-slot chirality of an arithmetic context. Operand-slot chirality records whether the evolving value occupies the first or second input of one binary operation. Chronological side records whether a new operator is composed before or after an existing product. In the convention (2), a future context acts on the left regardless of its operand slot. A theorem relating slot mirror, matrix transpose, inverse, or right continuation would require additional structure and is not assumed here.

3 Bivaluations and the projective quotient tower

The projective operator in Section 2 is already a condensation of a marked history. We now introduce a second, algebraic condensation: an operator is replaced by the image–kernel pair of a rank-one projector. No topology, norm, probability, or limiting dynamics is used in this section.

3.1 Points, copoints, and regular bivaluations

Let $V = K^2$ and let $V^\vee$ be its algebraic dual. A nonzero vector v defines a point $[v] \in \mathbb{P}(V)$, while a nonzero covector φ defines the projective line $\mathbb{P}(\ker \varphi) \in \mathbb{P}(V)$. The pair is transverse when $\varphi(v) \neq 0$.

Definition 3.1 (Regular bivaluation). A regular bivaluation over K is an ordered pair

$$(q^+, q^-) \in \mathbb{P}^1(K) \times \mathbb{P}^1(K), \quad q^+ \neq q^-.$$

The first member is called the image direction and the second the kernel direction. We write

$$\text{Biv}_K^{\text{reg}} := \mathbb{P}^1(K) \times \mathbb{P}^1(K) \setminus \Delta.$$

The order matters. Interchanging image and kernel replaces a projector by its complement.

Theorem 3.2 (Bivaluation–projector correspondence). *There is a natural bijection*

$$\text{Biv}_K^{\text{reg}} \longleftrightarrow \{\Pi \in \text{End}(V) : \Pi^2 = \Pi, \text{ rank } \Pi = 1\}.$$

If $q^+ = [v]$, $q^- = \mathbb{P}(\ker \varphi)$, and $\varphi(v) \neq 0$, the corresponding projector is

$$\Pi_{v,\varphi} = \frac{v \otimes \varphi}{\varphi(v)}. \quad (5)$$

Proof. For transverse lines L^+ and L^- one has $V = L^+ \oplus L^-$. There is a unique linear projection onto L^+ along L^- . Formula (5) gives it and is unchanged if v or φ is rescaled. Conversely, an idempotent splits $V = \text{im } \Pi \oplus \ker \Pi$. Rank one makes both summands distinct projective lines. The two constructions are inverse. $\square$

In the affine chart

$$v_a = \begin{pmatrix} -a \\ 1 \end{pmatrix}, \quad \varphi_b = \begin{pmatrix} -b & 1 \end{pmatrix},$$

one has $\varphi_b(v_a) = 1 + ab$ and hence

$$\Pi_{a,b} = \frac{1}{1+ab} \begin{pmatrix} ab & -a \\ -b & 1 \end{pmatrix}. \quad (6)$$

The divisor $1 + ab = 0$ is a genuine incidence failure of the transverse splitting. It must not be confused with the pole of one particular coordinate chart.

3.2 The homogeneous-space tower

Fix a reference ordered pair (q_0^+, q_0^-) and set

$$G = \mathrm{PGL}_2(K), \quad H = \mathrm{Stab}_G(q_0^+, q_0^-),$$

$$B_+ = \mathrm{Stab}_G(q_0^+), \quad B_- = \mathrm{Stab}_G(q_0^-).$$

Then $H = B_+ \cap B_-$. We use right cosets in G/H and $G/B_\pm$.

Theorem 3.3 (Projective quotient tower). *There are natural identifications*

$$G/H \cong \mathrm{Biv}_K^{\mathrm{reg}}, \quad G/B_+ \cong \mathbb{P}^1(K), \quad G/B_- \cong \mathbb{P}^1(K).$$

Under these identifications, the maps induced by $H \subseteq B_\pm$ forget respectively the kernel or image member of the ordered pair.

Proof. The group G acts transitively on ordered pairs of distinct projective points. The stabilizer of the reference pair is exactly H , so orbit–stabilizer gives G/H . The same argument for a single projective point gives the two Borel quotients. The descriptions of the forgetful maps follow from the subgroup inclusions. $\square$

Combining this result with projective evaluation gives the central algebraic tower

$$\boxed{\mathrm{Hist}_K \xrightarrow{\rho} G \xrightarrow{\pi_H} G/H \cong \mathrm{Biv}_K^{\mathrm{reg}} \longrightarrow G/B_\pm \cong \mathbb{P}^1(K).} \quad (7)$$

The first fiber consists of literal histories inducing one operator. The second consists of projective frames over one image–kernel result. These are different kinds of forgotten data.

3.3 A third point restores the projective frame

Definition 3.4 (Framed bivaluation). A framed regular bivaluation is a pairwise distinct ordered triple (q^+, q^-, q^∞) of points of $\mathbb{P}^1(K)$. The first two determine the rank-one projector; the third selects a projective frame above it.

Theorem 3.5 (Frame torsor). *After a reference triple has been chosen, the set of framed regular bivaluations is a G -torsor. The projection*

$$(q^+, q^-, q^\infty) \longmapsto (q^+, q^-)$$

has fibers on which H acts freely and transitively. Equivalently, $G \rightarrow G/H$ is an H -torsor.

Proof. A projective transformation is uniquely determined by its values on three distinct projective points. Hence G acts simply transitively on ordered projective frames. Holding the first two points fixed leaves exactly their stabilizer H , which acts simply transitively on the allowed third points. $\square$

For a Lie group this torsor may be equipped with a connection, but no such connection is canonical over an arbitrary field. A coordinate in H also requires a reference lift or local section.

Proposition 3.6 (Stabilizer-valued operator residue). *Let Π_0 be the projector of the reference bivaluation and define $\pi_H(g) = g\Pi_0g^{-1}$. Then*

$$\pi_H(g_1) = \pi_H(g_2) \iff g_2^{-1}g_1 \in H.$$

Thus two operator-level processes with the same condensed projector differ by an H -valued frame residue.

Proof. Equality of the two conjugated projectors is equivalent to

$$(g_2^{-1}g_1)\Pi_0(g_2^{-1}g_1)^{-1} = \Pi_0.$$

The stabilizer of a rank-one projector is precisely the stabilizer of its ordered image and kernel lines, namely H . $\square$

A static idempotent already splits $V = \text{im } \Pi \oplus \text{ker } \Pi$. It therefore does not carry a nontrivial linear extension class by itself. Process information discarded by $G \rightarrow G/H$ belongs instead to a lift, a connection and its holonomy, or some additional decoration of the history.

3.4 Finite-field calibration

Proposition 3.7 (Exact finite-field fiber counts). *For $K = \mathbb{F}_q$,*

$$|\mathbb{P}^1(\mathbb{F}_q)| = q + 1, \quad |\text{Biv}_{\mathbb{F}_q}^{\text{reg}}| = q(q + 1),$$

$$|\text{PGL}_2(\mathbb{F}_q)| = q(q^2 - 1), \quad |H| = q - 1.$$

Consequently every fiber of $\text{PGL}_2(\mathbb{F}_q) \rightarrow \text{Biv}_{\mathbb{F}_q}^{\text{reg}}$ has $q - 1$ elements.

Proof. There are $q + 1$ projective points and then q choices for a distinct second point. Moreover $|\text{GL}_2(\mathbb{F}_q)| = (q^2 - 1)(q^2 - q)$; quotienting by the $q - 1$ nonzero scalar matrices gives $q(q^2 - 1)$. The stabilizer of $(0, \infty)$ consists of $z \mapsto az$ with $a \in \mathbb{F}_q^\times$, and hence has order $q - 1$. $\square$

Example 3.8 (The field $\mathbb{F}_7$). One obtains

$$|G| = 336, \quad |G/H| = 56, \quad |H| = 6.$$

The 336 ordered projective frames condense to 56 ordered image–kernel pairs, with six frame lifts above each pair.

Corollary 3.9 (Uniform finite-frame information). *Let $G = \text{PGL}_2(\mathbb{F}_q)$ be uniformly distributed and let $Q = gH$. Then*

$$H(g \mid Q) = \log_2(q - 1).$$

This is the exact conditional information in the uniformly distributed frame coordinate above a bivaluation.

Proof. Every quotient fiber has $q - 1$ elements and the conditional distribution on that fiber is uniform. $\square$

The corollary is an algebraic information statement. Whether these bits are needed by an online computation depends on the continuation experiment of the next section.

3.5 Algebraic and dynamical condensation

Definition 3.10 (Algebraic condensation). Algebraic condensation is passage to an idempotent or quotient state, such as $g \mapsto gH$ or $g \mapsto g\Pi_0 g^{-1}$. It requires no limiting sequence.

Definition 3.11 (Dynamical condensation). Dynamical condensation is convergence, after a declared normalization, of a sequence of linear lifts to a singular rank-one operator, for example

$$\frac{\tilde{g}_n \cdots \tilde{g}_1}{r_n} \longrightarrow \Pi \quad \text{in } \text{End}(V).$$

It requires a topology or valuation, a choice of lifts, and a normalization.

Proposition 3.12 (No automatic dynamical condensation over a finite field). *In the discrete topology on $\text{PGL}_2(\mathbb{F}_q)$, every sequence of powers is periodic and convergence occurs only when the sequence is eventually constant. Thus the existence of rank-one idempotents over $\mathbb{F}_q$ does not imply a nontrivial attracting limit in G .*

The distinction will remain active throughout the paper. The quotient G/H is an exact algebraic state space; whether it is sufficient for future computation is a separate question.

4 Condensation and future continuation

A quotient forgets distinctions by definition. It becomes a computationally meaningful state only after a future action and an observation have been specified. This section gives the exact group-theoretic test for the projective quotient G/H .

4.1 Equivariant quotients and future sufficiency

Let a monoid M act on a set X from the left, let $q : X \rightarrow Q$ be a surjection, and suppose an action of M on Q satisfies

$$q(mx) = mq(x).$$

Let $o : Q \rightarrow Y$ be an observation. For $x, x' \in X$, define

$$x \equiv_{M,o} x' \iff o(mq(x)) = o(mq(x')) \quad \text{for every } m \in M.$$

Theorem 4.1 (Equivariant quotient as a residual state). *If $q(x) = q(x')$, then $x \equiv_{M,o} x'$. If the identity belongs to M and o is injective, then the converse also holds. Under those hypotheses the future-residual quotient of X is exactly Q .*

Proof. Equivariance gives $q(mx) = mq(x) = mq(x') = q(mx')$, and applying o proves the first statement. For the converse, test the identity continuation. Then $o(q(x)) = o(q(x'))$, and injectivity of o gives $q(x) = q(x')$. $\square$

The theorem does not say that every quotient is sufficient. It says that sufficiency follows from a declared descended action; minimality additionally uses an observation that can distinguish quotient states.

4.2 Left continuation on the bivaluation quotient

For the chronology convention of Equation (2), a future operator k acts on the current operator g by left multiplication: $g \mapsto kg$. The right-coset quotient therefore has the canonical action

$$k \cdot (gH) = (kg)H.$$

Corollary 4.2 (Exact left-continuation state). *Let all elements of G be allowed as left futures, include the identity future, and observe the resulting point of G/H through an injective map. Then two operators have the same future behavior exactly when they lie in the same right coset:*

$$g \equiv g' \iff gH = g'H.$$

Thus G/H is the canonical minimal exact state space for this continuation experiment.

In particular, the H -coordinate discarded by projective condensation is invisible to every left continuation whose observation factors through the same quotient.

4.3 Right continuation and the normalizer boundary

Opposite chronology replaces g by gk . Unlike left multiplication, this does not automatically act on right cosets.

Theorem 4.3 (Descent of one right update). *For a fixed $k \in G$, the formula*

$$R_k(gH) = gkH$$

defines a well-defined map $G/H \rightarrow G/H$ if and only if

$$k^{-1}Hk \subseteq H. \tag{8}$$

It is a bijection with inverse induced by k^{-1} if and only if $k \in N_G(H)$.

Proof. If $gH = ghH$ with $h \in H$, well-definedness requires $ghkH = gkH$. Cancelling gk on the left gives $k^{-1}hk \in H$ for every $h \in H$, which is (8). The same argument for k^{-1} gives the reverse inclusion. Both hold exactly when $k^{-1}Hk = H$, equivalently $k \in N_G(H)$. $\square$

Corollary 4.4 (Reversible right continuation). *A subgroup $L \leq G$ acts on G/H by right multiplication if and only if $L \subseteq N_G(H)$.*

For an infinite group, the inclusion in (8) may be strict, so the single-update and reversible statements must not be conflated. For finite H , equal cardinality turns the inclusion into equality.

Corollary 4.5 (Chronology-dependent visibility). *Suppose $h \in H$ is discarded by $g \mapsto gH$. Under left continuation one always has $kghH = kgH$, so h remains invisible. Under a right continuation k for which $k^{-1}hk \notin H$, the two representatives g and gh no longer determine the same quotient state after continuation. The discarded frame direction has become observable.*

This is the first rigorous bridge between projective process residue and a future-sensitive residual. The bridge is not the cardinality of H alone; it is the interaction between H and the declared continuation side.

4.4 The split-torus normalizer in rank one

Take the reference pair $(0, \infty)$. Then

$$H = \{z \mapsto az : a \in K^\times\}.$$

Let $J(z) = 1/z$; the sign is irrelevant projectively.

Proposition 4.6 (Normalizer of the ordered-pair stabilizer). *If $|K| > 2$, then*

$$N_G(H) = H \sqcup JH.$$

Equivalently, a projective map normalizes H exactly when it preserves the unordered set $\{0, \infty\}$; it either fixes the two points or interchanges them.

Proof. Because $|K| > 2$, choose $a \neq 1$. The map $z \mapsto az$ has fixed set exactly $\{0, \infty\}$. Hence the common fixed set of H is $\{0, \infty\}$. A normalizer sends this common fixed set to itself. If it fixes both points it lies in H ; if it swaps them, composition with J fixes both and lies in H . Conversely, H normalizes itself and $J(z) = 1/z$ conjugates $z \mapsto az$ to $z \mapsto a^{-1}z$. $\square$

Thus most right projective futures do not descend to the bivaluation quotient. Remembering only the rank-one projector is sufficient for the left chronology, but generally insufficient for arbitrary two-sided process composition.

4.5 Sufficient, exact, and minimal condensations

Definition 4.7 (Future-sufficient condensation). For a past space P , a continuation class M , and an observation Obs , a map $q : P \rightarrow Q$ is future-sufficient if there exist descended transitions on Q and an output map $o : Q \rightarrow Y$ such that every observation after every continuation can be computed from $q(p)$.

Definition 4.8 (Exact and minimal condensation). A future-sufficient condensation is exact when $q(p) = q(p')$ precisely for pasts indistinguishable by all allowed futures. It is minimal when every other exact deterministic state representation maps surjectively onto it.

Theorem 4.1 proves exactness for the projective left-action experiment. Section 5 gives the general residual construction and proves minimality without assuming a group quotient.

4.6 An exact-transport obstruction

A tempting way to label a path of states is to choose frames $h_t \in G$ and put

$$r_t = h_t h_{t-1}^{-1}.$$

This transport contains no independent edge information.

Proposition 4.9 (Exact edge labels telescope). *For $s < t$,*

$$r_t r_{t-1} \cdots r_{s+1} = h_t h_s^{-1}.$$

Every closed lifted path therefore has trivial product.

Proof. All intermediate factors cancel pairwise. $\square$

A nontrivial projective process holonomy consequently requires more than an endpoint potential. Possible additional data include an independent stabilizer-valued factor in each edge, a connection on $G \rightarrow G/H$, a multivalued frame, or rewrite 2-cells. None is supplied automatically by the static quotient.

5 Contextual residuals and minimal exact state

The projective quotient in Section 4 is one instance of a more general construction. A past is relevant only through the effects it can still have on legal futures. The definition must therefore name the interface, the future class, the observation, and the treatment of partial domains.

5.1 Typed interfaces and totalized partial action

Fix a typed interface C . Let P_C be the set of admissible pasts reaching that interface and let M_C be a monoid of future continuations. Composition is written as a right action $p \cdot \eta$ and may initially be partial because ordinary arithmetic, typing, or resource rules can make a continuation illegal. Before totalizing, require a domain-compositional partial action: $p \cdot (\zeta \eta)$ is defined exactly when $p \cdot \zeta$ and $(p \cdot \zeta) \cdot \eta$ are defined, and their values then agree. Execution of the free continuation word monoid has this property. Quotienting words by an operation relation requires a separate proof that the relation preserves both values and ordinary domains; projective cancellation alone does not supply one.

If the action is partial, introduce an absorbing state $\perp$ and put

$$P_C^\perp = P_C \sqcup \{\perp\}.$$

If the action is already total, use the convention $P_C^\perp = P_C$ and adjoin nothing. In the partial case, totalize the action by declaring an illegal continuation to have value $\perp$ and by setting $\perp \cdot \eta = \perp$. Let

$$O_C : P_C^\perp \longrightarrow Y_C^\perp$$

be the current observation, with $O_C(\perp) = \perp$ when the absorbing state is present and $\perp$ is distinct from every valid observation. Depending on the problem, O_C may record an endpoint, a projective state, a Boolean answer, a tensor, a trace, or a distribution.

Definition 5.1 (Contextual continuation equivalence). For $p, p' \in P_C^\perp$, define

$$p \equiv_C p' \iff O_C(p \cdot \eta) = O_C(p' \cdot \eta) \text{ for every } \eta \in M_C. \quad (9)$$

The quotient

$$\mathcal{R}_C = P_C^\perp / \equiv_C$$

is the contextual continuation-residual space at C .

The absorbing outcome is essential for partial arithmetic. Two pasts are not identified merely because all common legal continuations agree if one of them admits a future that the other does not.

Proposition 5.2 (Equivalence, refinement, and observation monotonicity). *The relation $\equiv_C$ is an equivalence relation. Enlarging the continuation class can only refine it. Replacing O_C by a coarser observation $f \circ O_C$ can only coarsen it.*

Proof. Reflexivity, symmetry, and transitivity hold pointwise for every continuation. A larger family supplies more equality tests. Applying a function to equal observations preserves equality and can identify previously distinct values. $\square$

Thus a residual is never absolute. It is relative to a declared experimental interface.

5.2 Residuals form the canonical continuation machine

Proposition 5.3 (Right congruence). *If $p \equiv_C p'$, then for every $\zeta \in M_C$,*

$$p \cdot \zeta \equiv_C p' \cdot \zeta.$$

Consequently the formula

$$[p] \cdot \zeta = [p \cdot \zeta]$$

defines a right action of M_C on $\mathcal{R}_C$.

Proof. For every further continuation η ,

$$O_C((p \cdot \zeta) \cdot \eta) = O_C(p \cdot (\zeta \eta)) = O_C(p' \cdot (\zeta \eta)) = O_C((p' \cdot \zeta) \cdot \eta).$$

□

The identity continuation shows that

$$\overline{O}_C([p]) = O_C(p)$$

is well defined. Hence $(\mathcal{R}_C, M_C, \overline{O}_C)$ is itself an exact deterministic continuation machine.

Definition 5.4 (Exact deterministic online realization). An exact deterministic online realization of the experiment at C consists of a state set S , a right action of M_C on S , an encoding $e : P_C^\perp \rightarrow S$, and an output map $o : S \rightarrow Y_C^\perp$ such that

$$e(p \cdot \eta) = e(p) \cdot \eta, \quad o(e(p)) = O_C(p). \quad (10)$$

The first identity includes the totalized invalid state when necessary.

The equivariance requirement prevents a representation from being called an online state merely because a separate external procedure knows the entire past and recomputes transitions from it.

Theorem 5.5 (Canonical minimal exact state). *For every exact deterministic online realization there is a unique surjective M_C -equivariant map*

$$\phi : e(P_C^\perp) \longrightarrow \mathcal{R}_C \quad \text{with} \quad \phi(e(p)) = [p].$$

It also preserves outputs. Therefore $\mathcal{R}_C$ is the canonical minimal reachable exact state space, up to isomorphism.

Proof. If $e(p) = e(p')$, then for every η equivariance and output correctness give

$$O_C(p \cdot \eta) = o(e(p) \cdot \eta) = o(e(p') \cdot \eta) = O_C(p' \cdot \eta).$$

Hence $p \equiv_C p'$, so ϕ is well defined. It is surjective by definition of $\mathcal{R}_C$. Equivariance follows from Proposition 5.3, and output preservation follows from the identity continuation. Uniqueness is forced on every reachable state $e(p)$. □

This is the finite-state principle behind the classical theory of residual languages and automata [7], but the present formulation permits arbitrary typed observations and partial arithmetic domains.

5.3 Finite-state information lower bound

Corollary 5.6 (Fixed-width exact-state bound). *Assume $\mathcal{R}_C$ is finite. If an exact reachable state is stored in a fixed b -bit register, then*

$$b \geq \lceil \log_2 |\mathcal{R}_C| \rceil. \quad (11)$$

The same conclusion holds for a prefix-free variable-length encoding whose maximum codeword length is b .

Proof. Theorem 5.5 requires at least $|\mathcal{R}_C|$ reachable code states. A fixed b -bit register has at most 2^b states. For a prefix-free code of maximum length b , Kraft's inequality also gives at most 2^b codewords. $\square$

The theorem is a state-selection lower bound. It does not determine the size of a transition table, the cost of computing the encoding, the time needed to apply a continuation, or the materialized size of a structured value.

Corollary 5.7 (Fixed-register capacity–time bound). *Consider declared cuts $C_0, \dots, C_T$. Suppose that at cut C_t a complete exact state for every admissible past must fit in a fixed-width register of capacity b_t bits, and charge one full register snapshot at each cut. Then*

$$\sum_{t=0}^T b_t \geq \sum_{t=0}^T \lceil \log_2 |\mathcal{R}_{C_t}| \rceil. \quad (12)$$

If $|\mathcal{R}_{C_t}| \geq ct^d$ eventually, the right side is $\Omega(T \log T)$; if $|\mathcal{R}_{C_t}| \geq ca^t$ with $a > 1$, it is $\Omega(T^2)$.

Proof. Apply Corollary 5.6 at each cut and sum. The two asymptotic statements follow from summing $\log t$ and t , respectively. $\square$

Equation (12) is deliberately called a capacity–time bound. It becomes an actual memory–time bound for a run only when the operational model charges the complete register as live memory at each snapshot. Variable-size structured states, compression, communication, and recomputation require the model of Section 7.

5.4 Three projective residual experiments

The quotient tower supplies three different experiments.

1. If pasts are operators $g \in G$, futures act on the left, and the observation is the bivaluation gH , then $\mathcal{R} \cong G/H$ by Corollary 4.2.
2. If the observation remembers only gq_0^+ and futures continue to act on the left, then the residual is G/B_+ . The kernel direction has become observationally irrelevant.
3. If arbitrary reversible futures act on the right, G/H does not support the action unless the futures lie in $N_G(H)$. A full operator lift, or another state carrying the missing frame information, is then required.

Thus projective process residue can coincide with a continuation residual, map to one, or be necessary to refine one. Which case occurs is a theorem about the chosen action and observation, not a property of the static fiber alone.

5.5 Randomized and approximate boundaries

Definition 5.1 uses exact deterministic equality. A randomized algorithm may permit equal output distributions rather than equal outputs; an approximate algorithm may identify observations within a metric or loss tolerance; a statistical model may retain only a sufficient statistic for a declared distribution family. Each change alters the residual relation and its lower bound. No such approximation theorem is inferred from Corollary 5.6; approximate residual geometry remains an open programme.

6 Ported AES: substitution, execution, and composition

The residual theory specifies when a state retains enough information for future computation. We now give a finite program interface that can be attached to a punctured AES without identifying its geometry with its program. Paper 0 owns the carrier geometry and its compactification [1]; Paper I supplies marked one-hole histories [11]. The construction here is a structural proposal for ported realizations. Its finite-network and descent results are proved with the hypotheses stated below. The program/source distinction follows the interface discipline of the Adva program–process core [15].

6.1 Three kinds of opening

An essential geometric puncture is a declared end where the given assignment and metric do not extend as regular AES data. A computational hole is an ordered place for program substitution. A port is an input or output interface associated with a geometric end or a specified regular interface location. Their identification requires additional data.

Definition 6.1 (Ported AES realization proposal). Let $E = (M, g, a; \mu, \lambda)$ be an oriented regular AES with a declared compactified carrier and marked end set S , where $|S| = k$. A ported realization consists of ordered typed port families I, O , an anchoring map

$$\pi : I \sqcup O \longrightarrow S \sqcup L,$$

where L is a declared set of regular interface locations, a finite typed program decoration, and source, occurrence, boundary, and assembly records. The map π is not assumed injective or surjective. Ports at a puncture carry data in its end interface; they are not regular points with a value $a(p)$.

The three integers $k, |I|, |O|$ are independent. A three-output experiment declares $|O| = 3$; its internal input frontier may have any finite size. If all ports anchor injectively into S , then $k \geq |I| + |O|$; equality follows for a bijection. Ends without ports and ends carrying several ports are allowed. The roles of the outputs must be declared. They are not automatically three scalar values or Adva’s construction/space/time views; the outer interface and the internal frontier are distinct [14].

Input binding retains the AES geometry unless a separate parameter family (g_θ, a_θ) is supplied. Such a family needs a proof of the AES identity, its admissible parameter domain, and its marked end behavior. Binding is also distinct from topological capping.

6.2 Finite programs, sources, and occurrences

Definition 6.2 (Strict finite arithmetic network). An open network $P : A \rightsquigarrow B$ is a finite directed acyclic graph with ordered typed input and output boundaries and ordered input/output slots at every gate. Each wire has one producer and one consumer, counting an external output as

a consumer. Copy and discard are explicit gates. Primitive gates denote deterministic partial functions. Source identities are immutable; distinct uses have distinct occurrence identities, even if their sources or values agree. External boundary positions likewise have occurrence identities separate from source identities.

The arithmetic fragment used below has exact rational constants, addition, subtraction, multiplication, division, explicit copy, and strict discard. Strict discard requires its input producer to be defined. Thus $\text{discard}(1/0)$ remains outside the domain. A finite acyclic network cannot exhibit feedback divergence; feedback, side effects, and unbounded interpretation need separately declared semantics.

Repeated use of a program template first creates separate named instances. Their event occurrences are fresh; their original sources remain the same. Administrative wire and event renaming then acts on those instances without changing source or occurrence identity. Here a source label names the declared syntactic origin of a gate or wire occurrence. The ancestry of a value supplied after substitution is separately recovered from the connected producer DAG and binding receipt; binding does not overwrite a body's immutable source labels. Template reuse is therefore distinct from implicit sharing of one produced wire.

6.3 Program substitution and its receipts

Let P have ordered holes $i_1, \dots, i_m$, and let $q_1, \dots, q_r$ be already instantiated producer programs. A binding β matches producer output occurrences to selected holes, preserving types and positions. Every supplied output is used once, and every selected hole is filled once. Unused outputs require an explicit discard; multiple uses require an explicit copy. Holes not selected by a partial binding remain on the open frontier. Fix the external order as producer inputs in argument order, followed by the unfilled body inputs. Sequential partial filling with open producers can induce a different order from simultaneous filling; comparison then requires a declared typed boundary permutation. The closed-producer example below requires none.

Construction 6.3 (Certified simultaneous substitution). Take disjoint administrative copies of the body and producer graphs, identify the matched boundary wires, and retain all primitive gate occurrences and sources. Record the ordered binding, producer regions, body region, and call nesting in a graft receipt. The resulting network is

$$\text{Subst}_\beta(P; \mathbf{q}).$$

Accept it only when the resulting global graph is acyclic and its boundary inventory is complete. A numerical input is admitted by an explicit constant producer with its source record, rather than by erasing the producing process.

For ordinary one-way substitution, producers depend only on their external inputs and the body depends on their outputs, so acyclicity follows. General seam assembly has no such guarantee: two locally acyclic pieces can form a cycle after cross-wiring. The notation Subst is reserved for program substitution; Fill in Section 9 continues to denote rewrite filling area.

Proposition 6.4 (Type and identity preservation). *A legal substitution preserves boundary typing, primitive gate slot order, source identities, and instantiated gate occurrences. Distinct occurrences are not identified by equality of numerical value.*

Proof. The disjoint copies separate administrative names. The binding changes only which wire supplies a selected body input; it creates no primitive gate and deletes none. Its injectivity on

producer output occurrences and selected holes preserves the single-consumer rule. Source and occurrence records are transported by the stated renaming map. The remaining boundaries retain their declared order. All other identifications are excluded by construction. $\square$

6.4 Substitution and partial evaluation

Write D_P for the strict numerical domain of P : inputs for which every primitive gate required by the network is defined, including producers eventually consumed by discard. Primitive numerical functions are pure; no primitive observes the scheduler. The numerical result is therefore independent of the chosen topological schedule. An operational trace, its first observed failure, and its costs can still depend on that schedule.

Theorem 6.5 (Substitution–evaluation compatibility). *Let $Q : X \rightsquigarrow A$ and $P : A \times Z \rightsquigarrow B$ be compatible strict finite networks. For the binding consuming all outputs of Q , their composite has domain*

$$D_{P \circ Q} = \{(x, z) : x \in D_Q, (\text{Eval}(Q)(x), z) \in D_P\}. \quad (13)$$

On exactly this domain,

$$\text{Eval}(P \circ Q)(x, z) = \text{Eval}(P)(\text{Eval}(Q)(x), z). \quad (14)$$

The same statement applies to finitely many disjoint producers and partial filling, retaining the unfilled boundary.

Proof. Use a topological order with producer gates before body gates. A producer gate has precisely the same ordered inputs, primitive partial function, and domain predicate as in Q . Induction gives the same values on its output wires whenever the producer is defined. The bound body inputs are therefore exactly $\text{Eval}(Q)(x)$. A second topological induction through the body gives its original gate values and domain predicates. Every gate of the composite is consequently defined if and only if both conditions in Equation (13) hold. This proves both equations. Purity permits a different topological order without changing the successful result or whether all gates are defined; it does not identify ordered traces. $\square$

Corollary 6.6 (Three equality levels for composition). *Compatible composition has the following laws.*

1. *Partial numerical functions satisfy identity and associativity strictly, including equality of their domains.*
2. *Operation networks satisfy these laws up to an explicit isomorphism of administrative names and contraction of administrative port segments. The isomorphism preserves instantiated occurrences, sources, types, slot orders, and ordered external boundaries. Identity networks use the same source and occurrence contract as the interface object they preserve.*
3. *Literal receipts for different calls or parenthesizations remain distinct. Their final operation networks are related by the preceding transport certificate, not by deleting the call history.*

Proof. Equation (13) applied twice gives the same three domain conditions for either parenthesization; Equation (14) then gives the same result. At graph level, both constructions contain the same instantiated primitive gates and the same final boundary identifications. Map each renamed gate and wire to its counterpart and contract only administrative segments. This supplies the stated isomorphism. A receipt records the actual calls and their nesting, which these operations do not make equal. $\square$

6.5 Joint geometric and computational descent

Proposition 6.7 (Collar and interface descent). *Suppose finitely many oriented regular AES pieces have the same signed parameters (μ, λ) , and orientation-compatible open collar maps ϕ_{vw} produce a Hausdorff smooth surface. Require inverse and cocycle conditions and, on every overlap,*

$$a_v = a_w \circ \phi_{vw}, \quad g_v = \phi_{vw}^* g_w.$$

Require computational seam records to agree on ordered ports, types, sources, occurrences, gate parameters, and boundary wire identities, with a complete assembly record for nonlocal relations. Require the assembled global program to be a finite DAG. Require marked end identities and the anchoring map π to be compatible with all identifications.

Then the AES data and the computational network descend together. The canonical arithmetic frames transform by the collar Jacobians. The assembled program obeys Theorem 6.5 for every compatible causal composition across its interfaces.

Proof. Overlap equality descends the smooth function and positive definite metric. The identity $|da|_g^2 = \mu^2 + \lambda^2 a^2$ is tensorial and holds in every local chart, hence globally. Compatible orientation and equal signed parameters give the same canonical frame formula in each chart, so those frames also descend. The seam inventory identifies precisely the specified wire occurrences without merging sources or introducing implicit copies. Compatible end markings and anchoring maps descend the attachment of ports to the carrier. Global acyclicity yields a topological execution order. The numerical composition result follows from the finite-network theorem. $\square$

Warning 6.8 (The motion bridge remains additional). Proposition 6.7 does not prove that a gate of the attached network is an AES motion. Such a realization requires primitive partial state transformations Φ_ω , exact domains, parameter/source transport, and overlap equations

$$T_{vw} \circ \Phi_\omega^v = \Phi_\omega^w \circ T_{vw}$$

on their stated domains. Binary gates, explicit copy/discard, and negative scalings cannot be inferred from the single-value positive affine model. Full-state tangential residuals must also be retained: an identity arithmetic word on assignment values need not be the identity on AES states. Attaching a program to a surface does not establish a faithful arithmetic-motion realization.

For an already given genus-zero k -end carrier, $k \geq 3$, a pants decomposition has $k - 2$ three-end pieces and $k - 3$ internal seams. Restrict all data and retain the overlap and assembly records. Joint descent then recovers the original ported object. A second decomposition gives another atlas of that same object. Each three-end piece may have punctures and seam boundaries; it is not thereby a complete three-cusp AES. Conformal point compactification additionally requires punctured-disc ends. These geometric constructions and distinctions belong to Paper 0 [1].

Local block frontiers can alternate across a seam. Even when the global event graph is acyclic, the coarse graph whose vertices are whole geometric pieces may contain a cycle. One then cannot execute each whole piece just once in a block order; execution uses the global causal event order and the exposed seam frontier.

6.6 Features and future computation

Fix the experiment (P_C, M_C, O_C) of Section 5, with its declared treatment of illegal arithmetic. Use a domain-compositional partial action, or the free continuation word monoid, before totalization; do not import unproved operation relations. Divergence is not identified with a type error or division

by zero. Here $\cdot$ is a declared total right action on $P_C^\perp$, for example execution of continuation words with an absorbing illegal state and a distinct invalid observation. Contextual equivalence means

$$p \equiv_C p' \iff O_C(p \cdot \eta) = O_C(p' \cdot \eta) \text{ for every } \eta \in M_C.$$

A right congruence is an equivalence relation preserved by every continuation.

Let $\chi_C : P_C^\perp \rightarrow Z_C$ be a proposed feature. It may retain a scalar, operator, marking, source/occurrence package, or residual. The kind of feature and its observation policy are part of the interface.

Theorem 6.9 (Feature observation descent). *The following conditions are equivalent.*

1. *For every declared continuation η , there is a function o_η on the reachable feature image satisfying*

$$o_\eta(\chi_C(p)) = O_C(p \cdot \eta).$$

2. *$\ker \chi_C$ is contained in contextual continuation equivalence:*

$$\chi_C(p) = \chi_C(p') \implies p \equiv_C p'.$$

Legal-domain differences are included through the declared invalid outcome.

Proof. The first condition makes all future observations equal on each feature fiber, which is exactly the second condition. Conversely, define o_η using any representative of a feature fiber; the second condition makes the choice immaterial. This is existence of observation factorization, not an assertion that the factors can be computed efficiently or at all. $\square$

Proposition 6.10 (Online feature update). *Feature transitions u_η satisfying*

$$u_\eta(\chi_C(p)) = \chi_C(p \cdot \eta)$$

exist on the reachable image if and only if $\ker \chi_C$ is a right congruence. Together with factorization of O_C , they give an exact feature continuation machine. An executable implementation still requires effective transition and observation maps.

Proof. Necessity follows by applying u_η to equal feature values. For sufficiency, define u_η by any representative. Right congruence makes it well defined, and the action laws descend from the given action. Output factorization supplies the current observation. $\square$

Example 6.11 (Observation sufficiency without online update). Take states $0, 1, 2$, constant observation zero, and the idempotent update $f(0) = 0, f(1) = 2, f(2) = 2$. Every state is contextually equivalent for the monoid $\{\text{id}, f\}$. Let $\chi(0) = \chi(1) = 0, \chi(2) = 1$. Feature observation descent holds, but $\chi(f(0)) \neq \chi(f(1))$, so no feature update can be defined on input feature zero. A future-sufficient observation and a closed online state are different requirements.

Actual opposite-edge characteristic extraction remains a separate witness problem [16]. If a spectral feature is used, first declare the endomorphism closure, operator domain, marking, and residual. A pairwise relation does not automatically have an ordinary spectrum. Three-side feedback also requires a snapshot and update schedule or a separately proved fixed-point semantics.

6.7 A four-puncture, four-input, three-output experiment

Example 6.12 (Fixed AES carrier with program interfaces). On $M = \mathbb{P}^1(\mathbb{C}) \setminus \{-1, 0, 1, \infty\}$, put

$$\begin{aligned} a &= \frac{x^4}{4} - \frac{x^2}{2} + y^2, & A &= x^3 - x, & B &= 2y, \\ L &= A^2 + B^2, & F &= 1 + a^2, & g &= \frac{L}{F}|dz|^2. \end{aligned}$$

Off the three finite deleted points, $L > 0$ and $|da|_g^2 = F$. At each finite point $da = 0$, excluding a nondegenerate regular AES extension. Along the imaginary axis $a \rightarrow \infty$, excluding a real-valued extension at infinity. Thus all four marked ends are essential. This direct verification does not use a general puncture/critical-point assertion.

Anchor the four input holes to $-1, 0, 1, \infty$, in that order, keeping input values as port data. The finite assignment limits $(-1/4, 0, -1/4)$ and the divergent infinite limit are not replaced by those values. Anchor the three output ports at $i, 1+i, -1+i$, respectively; their output values likewise need not equal the assignment values at those regular sites. Choose producers

$$q_1 = 1 + 1 = 2, \quad q_2 = 3 + 4 = 7, \quad q_3 = 6/2 = 3, \quad q_4 = 9 - 4 = 5.$$

The body, with an explicit copy for each twice-used input, reads out

$$y_1 = x_1 + x_2, \quad y_2 = x_3 x_4, \quad y_3 = \frac{x_1 + x_3}{x_2 - x_4}.$$

The output is exactly $(9, 15, 5/2)$. It is a declared scalar projection of this experiment, not an identification with Adva's native three output roles. The run retains eight constant sources, their occurrences and explicit copy records, the operation DAG, and the graft receipt.

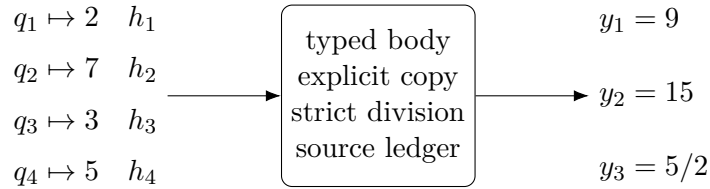


Figure 2: A fixed four-end AES carrier with four computational inputs and three declared scalar outputs. The diagram displays the program interface, not paths of arithmetic gates through the AES.

Use either cut $|z - c| = 3/4$ with $c = -1/2$ or $c = 1/2$. The inner and outer charts on the overlap are

$$\xi = \frac{z - c}{3/4}, \quad \eta = \frac{3/4}{z - c}, \quad \eta = 1/\xi.$$

Each cut separates two marked ends from the other two. For each cut, also declare an allocation of gate occurrences to local packets; this allocation is additional program data, not a consequence of the port anchoring map. Restrict the carrier data to the chosen pieces and retain each local event region, its exposed wire frontier, and the complete ordered seam/assembly contract. Reciprocal coordinate transport is an exact change of chart. The tensor and chain-rule argument in Proposition 6.7 recovers the same global AES and operation network from either decomposition. Local seam source references identify transported data; they are not new constant sources. Execution after assembly follows the global event order, which need not be a whole-piece block order. No path of a binary gate through the AES is asserted.

6.8 Finite checks and remaining obligations

The accompanying independent finite checker uses exact rational arithmetic for this example and explicit positive and negative controls. Its scope and reproduction commands are recorded in the Paper IV README. It is a new verification of the integrated model, not a rerun of the unavailable checker reported with the candidate chapter. Finite checks supplement the proofs; they do not prove continuous collar identities or a faithful motion bridge.

The important negative controls are implicit sharing, duplicate binding, mismatched types, unaccounted seam data, and cycles formed by locally acyclic pieces. A literal constant two and the producer $1 + 1$ have the same scalar output but different sources and operation histories. Strict evaluation retains failure of a discarded $1/0$ producer. Different association receipts remain distinct, and Example 6.11 remains an obstruction to online feature update.

The remaining obligations are stated as Problems 14.3 and 14.4: realize the actual typed generators by AES state transformations and construct the actual opposite-edge feature witnesses. Neither is a consequence of finite program substitution.

7 Operational resource geometry

Contextual residuals measure semantic distinguishability. An execution uses concrete data structures, schedules, memories, and communication links. The two layers meet only after a machine model says where a complete residual state is represented. The strict networks of Section 6 supply one concrete class of typed acyclic programs. Their substitution theorem preserves partial numerical semantics and provenance; it does not preserve the work, storage, or scheduling cost of a chosen implementation.

7.1 Logical cuts and live configurations

Let Γ be a typed acyclic computation network. A logical cut separates operations already licensed by causality from operations not yet licensed. A set of nodes that have ever been computed may record causal progress, but it does not say which values are currently present.

Definition 7.1 (Operational machine model). An operational model $\mathcal{M}$ specifies:

1. the admissible input family and uniformity convention;
2. value, instruction, and constant encodings;
3. atomic compute, erase, move, communicate, checkpoint, and recompute actions;
4. legal live configurations and their sizes;
5. the cost of each action and the processor or memory topology;
6. whether input, output, constants, tables, and code count toward space.

An execution is a legal trace

$$L_0 \xrightarrow{a_1} L_1 \xrightarrow{a_2} \dots \xrightarrow{a_T} L_T, \quad (15)$$

where L_t lists the values and control records currently materialized. Write $S_\sigma(t) = \text{size}_{\mathcal{M}}(L_t)$.

Definition 7.2 (Resource vector). For a realization Γ and schedule σ , define

$$\mathbf{C}_{\mathcal{M}}(\Gamma, \sigma) = (L_{\text{desc}}, W, T, D_{\text{caus}}, S_{\text{max}}, \text{ST}, Q), \quad (16)$$

where

$$S_{\max} = \max_t S_{\sigma}(t), \quad \text{ST} = \sum_{t=0}^T S_{\sigma}(t).$$

Here L_{desc} is static description size, W weighted work, T scheduled time, D_{caus} intrinsic causal depth in the declared parallel model, and Q communication or I/O.

These coordinates generally have different units. A theorem comparing them must state the simulation, charging, and encoding convention; adding them into one scalar without such a rule has no invariant meaning.

7.2 Why completed-node sets do not determine space

Theorem 7.3 (Completed-set insufficiency). *Suppose a computation has a positive-size intermediate value that may legally be retained or erased after its last immediate use. Then the set of nodes ever computed does not determine the current workspace.*

Proof. Execute through the last use of the intermediate. In one legal execution keep its value; in another erase it. The two executions have computed exactly the same nodes, but their live configurations differ by a positive-size object. $\square$

With recomputation the discrepancy is stronger: a value may disappear and later be recreated while the ever-computed set remains unchanged. Therefore a monotone chain of down-sets cannot serve as a general pebble or checkpoint state.

Proposition 7.4 (Restricted frontier model). *In a no-recomputation node-value model in which each value is retained exactly until its last successor has executed, a down-closed completed set I does determine the live frontier*

$$\partial^+ I = \{u \in I : \text{some successor of } u \text{ lies outside } I\}.$$

If node u occupies $b(u)$ units, then the corresponding workspace is $\sum_{u \in \partial^+ I} b(u)$, up to the separately declared input, output, and control conventions.

The proposition remains useful, but only inside its restricted retention policy.

7.3 Embedding the residual lower bound

A live configuration may carry much more than the minimal residual class. It may include redundant values, caches, checksums, code pointers, or a convenient structured representation. Nevertheless, when it is the complete exact online state, it must distinguish residual classes.

Proposition 7.5 (Live-state residual bound). *At a cut C , suppose the charged live configuration contains a complete exact state and is encoded either in a fixed-width b -bit slot or by a prefix-free code of maximum length b . Then*

$$b \geq \lceil \log_2 |\mathcal{R}_C| \rceil.$$

Proof. The live-state encoding is an exact online realization, so Theorem 5.5 maps its reachable states onto $\mathcal{R}_C$. Apply the counting argument of Corollary 5.6. $\square$

The following quantities must remain distinct:

Quantity	Question answered
$ \mathcal{R}_C $	How many future behaviors remain semantically distinguishable?
$\lceil \log_2 \mathcal{R}_C \rceil$	How many bits select one exact residual class in a fixed-width model?
$S_\sigma(t)$	How large is the materialized configuration in this execution?
L_{desc}	How large is the program, circuit, table, or shared representation?

A dense tensor with D field entries, a low-rank factorization of the same tensor, an index into a decision diagram, and the whole decision diagram are four different representations. Their sizes are not determined by the cardinality of one abstract residual space.

7.4 Uniform families and Pareto resource geometry

Complexity belongs to input families rather than to one hard-coded finite map. Let $F = (F_n)_{n \geq 1}$ be a family and let $\text{Adm}_{\mathcal{M}}^{\text{unif}}(F)$ be the realization families produced by one declared uniform constructor. Define the fixed-size achievable frontier

$$\mathfrak{C}_{\mathcal{M}}^{\text{unif}}(F; n) = \text{Pareto} \left\{ \mathbf{C}_{\mathcal{M}}(\Gamma_n, \sigma_n) : ((\Gamma_j, \sigma_j))_{j \geq 1} \in \text{Adm}_{\mathcal{M}}^{\text{unif}}(F) \right\}. \quad (17)$$

Here Pareto retains nondominated vectors under coordinatewise comparison. If limits are not attained, a closure may be needed.

Equation (17) is a structural framework, not a machine-independent invariant. Its value changes with the model, units, precision, uniformity, preprocessing, and allowed representations. A robust comparison across models would require explicit efficient simulations.

7.5 Three distinct order effects

The examples of this paper separate three phenomena:

Order effect	Meaning	Calibration
semantic noncommutation	Order changes the induced operator or endpoint	affine add-scale histories
rewrite-plan variation	Semantics agree but factorization and costs differ	matrix-chain parenthesization
schedule or cut sensitivity	One computation exposes different live or residual widths	OBDD order and checkpointing

Only the first already has the contact-curvature interpretation developed in Paper 0 [1]. The other two should not be called curvature until a connection, gauge law, or 2-cell defect has been supplied.

8 Word growth, fibers, and information loss

A quotient fiber can grow with word length, but cardinality, information, and running time are different measurements. This section records the exact counting statements that survive without a machine model.

8.1 Words, image growth, and fiber multiplicity

Let Σ be a finite alphabet with $s = |\Sigma| \geq 2$. For each n , let

$$\rho_n : \Sigma^n \longrightarrow Y_n$$

be an evaluation or condensation map. Define

$$N_n = |\rho_n(\Sigma^n)|, \quad M_n = \max_{y \in Y_n} |\rho_n^{-1}(y)|.$$

Proposition 8.1 (Fiber–image inequality). *For every n ,*

$$M_n \geq \frac{s^n}{N_n}. \quad (18)$$

Proof. The s^n words are partitioned among N_n nonempty fibers. At least one fiber has size at least the average. $\square$

The inequality does not require a group, metric, or probability model. It shows only that small image growth forces some large history fiber.

8.2 A conditional-information form

Let W_n be uniformly distributed on Σ^n and put $Y_n = \rho_n(W_n)$. Shannon entropy is measured in bits [9].

Proposition 8.2 (Evaluation information loss). *One has*

$$H(W_n | Y_n) = n \log_2 s - H(Y_n) \geq n \log_2 s - \log_2 N_n, \quad (19)$$

and

$$H(W_n | Y_n) \leq \log_2 M_n.$$

Proof. Because Y_n is a deterministic function of W_n , $H(W_n, Y_n) = H(W_n) = n \log_2 s$, while the chain rule gives $H(W_n, Y_n) = H(Y_n) + H(W_n | Y_n)$. Also $H(Y_n) \leq \log_2 N_n$. Conditional on $Y_n = y$, the word lies in a set of size at most M_n , so its conditional entropy is at most $\log_2 M_n$; averaging preserves the bound. $\square$

For a nonuniform distribution, the lost information depends on the fiber weights, not only their cardinalities. For an individual word, Kolmogorov or description complexity would introduce another model and is not being used here.

8.3 Word metrics and coarse growth

Let G be a finitely generated group with finite symmetric generating set S . Write $|g|_S$ for word length and

$$\beta_S(n) = |\{g \in G : |g|_S \leq n\}|.$$

Proposition 8.3 (Finite generating sets give comparable growth). *If S and T are finite generating sets, there are constants $A, B \geq 1$ such that*

$$|g|_T \leq A|g|_S, \quad |g|_S \leq B|g|_T$$

for all $g \in G$. Consequently

$$\beta_T(n) \leq \beta_S(Bn), \quad \beta_S(n) \leq \beta_T(An).$$

Proof. Every generator in one finite set is a bounded-length word in the other. Let A and B be the corresponding maximum lengths and substitute each letter in a shortest word. $\square$

Polynomial versus exponential group growth is therefore a coarse property of the finitely generated group, not of one particular generating set. Deep structure theorems classify polynomial growth [6]; the present paper needs only the elementary comparison above.

8.4 Four growth functions that must not be merged

For a history language and evaluation map one may encounter:

$$\begin{aligned} W(n) &= \#\{\text{literal words of length at most } n\}, \\ N_\rho(n) &= \#\{\text{distinct evaluated operators reached by those words}\}, \\ M_\rho(n) &= \max_g \#\{\text{words of length at most } n \text{ evaluating to } g\}, \\ R_C(n) &= \#\{\text{contextual residual classes exposed at a selected cut}\}. \end{aligned}$$

A search algorithm adds still another quantity: the number of states it actually explores under a specified strategy. None of these functions is the running time by definition.

Example 8.4 (Free reduction). The ball of radius n in a free group of rank at least two grows exponentially, but a given word can be freely reduced by a stack in time linear in its input length. Exponential group growth therefore does not imply exponential evaluation time.

Example 8.5 (Finite projective image). For fixed q , every sufficiently long bilateral word over a finite generating alphabet evaluates into the finite group $\text{PGL}_2(\mathbb{F}_q)$. Literal word counts continue to grow, while $N_\rho(n)$ eventually saturates at at most $q(q^2 - 1)$. The growing fibers measure many realizations of a finite semantic state, not unbounded state-space size.

8.5 Geometry requires a comparison map

Hyperbolic spaces and many groups have exponential ball growth. A history language may also have exponential word growth. Equality of the adjectives “exponential” does not provide a quasi-isometry, coding theorem, or simulation. A geometric complexity theorem would need at least:

1. a map from histories or live states into the geometric space;
2. upper and lower distortion estimates for the selected metrics;
3. a simulation relating geometric steps to machine actions;
4. a representation theorem linking geometric volume to encoded states.

Without these data, curvature and volume remain motivating analogies. The fixed-model identities in the case studies below are not promoted to a general curvature–complexity law.

9 Rewrite fibers, normal forms, and exactness obstructions

Evaluation fibers contain realizations with equal semantics. A rewrite system organizes paths inside such a fiber. The cost of traversing a rewrite path, the distance between two realizations, the filling of a coherence loop, and a possible holonomy are distinct structures.

9.1 Rewrite graphs and normal forms

Let X be a set of realizations and $\rho : X \rightarrow Y$ their semantics. For $y \in Y$, write

$$\mathcal{F}_y = \rho^{-1}(y).$$

A semantics-preserving rewrite relation $\mathcal{R}$ has directed edges $x \rightarrow x'$ only when $\rho(x) = \rho(x')$. It therefore decomposes into graphs on the fibers $\mathcal{F}_y$.

Definition 9.1 (Rewrite distance and filling). For two vertices in the same undirected rewrite component, let $d_{\mathcal{R}}(x, x')$ be the shortest number or declared weight of rewrite edges connecting them. If rewrite relations are supplied as 2-cells, let $\text{Fill}_{\mathcal{R}}(\ell)$ be the minimum number or weight of 2-cells filling a closed rewrite loop ℓ , when such a filling exists.

A terminating and confluent rewrite system selects a unique normal form in each connected semantic class. Termination plus local confluence is sufficient for confluence by Newman's lemma [8]. A normal form may make equality testing efficient, but its construction cost and output size still require a computational model.

9.2 Potential labels are exact

Let A be an additive abelian group and let $C : X \rightarrow A$ be any single-valued potential, for example a cost assigned to a completed realization.

Proposition 9.2 (Additive exactness obstruction). *If a rewrite edge is labelled*

$$\omega_C(x \rightarrow x') = C(x') - C(x),$$

then every finite closed rewrite loop satisfies

$$\sum_{e \in \ell} \omega_C(e) = 0.$$

Proof. The sum telescopes; every intermediate potential occurs once with each sign, and the initial and terminal vertices coincide. $\square$

The same obstruction holds noncommutatively for pure-gauge labels.

Proposition 9.3 (Group-valued pure-gauge obstruction). *Let $h : X \rightarrow L$ take values in a group and label an edge by*

$$r(x \rightarrow x') = h(x')h(x)^{-1}.$$

With chronological multiplication, the ordered product around every closed loop is the identity.

Proof. For a path $x_0 \rightarrow x_1 \rightarrow \cdots \rightarrow x_m$, the chronological product is

$$r(x_{m-1} \rightarrow x_m) \cdots r(x_0 \rightarrow x_1) = h(x_m)h(x_0)^{-1}.$$

The factors cancel successively. On a closed loop $x_m = x_0$, this is the identity. $\square$

These propositions apply to endpoint-potential differences on an associahedron and to the exact frame transport of Proposition 4.9. They do not say that every connection is flat; they say that a globally exact label cannot certify nontrivial holonomy.

9.3 Action is not holonomy

A closed rewrite plan may have positive accumulated work even when its exact potential sum and pure-gauge product vanish. Work charges actions, whereas holonomy compares transported frames after quotienting by gauge. Calling positive loop work “curvature” would erase this distinction.

For a weighted directed rewrite path $x = x_0 \rightarrow x_1 \rightarrow \cdots \rightarrow x_m = x'$, define

$$\text{Act}(x_\bullet) = \sum_{i=0}^{m-1} c(x_i \rightarrow x_{i+1}), \quad c \geq 0.$$

Even when $x = x'$, the action can be positive. This records dissipation or work under the selected rewriting algorithm, not a group-valued failure of exact transport.

9.4 Large fibers can have shared representations

A semantic fiber may contain exponentially many words while admitting a small DAG, grammar, circuit, or congruence-closed representation that shares common substructure. Conversely, a small semantic set can require large intermediate objects in a restricted representation language. Therefore

$$|\mathcal{F}_y| \text{ large} \not\Rightarrow L_{\text{desc}} \text{ large}$$

without a lower-bound theorem for the chosen representation class.

The case studies make the separation concrete:

- Horner words have an exactly countable charge fiber of distinct operators;
- OBDDs share equal residual functions but can still have exponentially different widths under different variable orders;
- matrix-chain parenthesizations are vertices of an associahedral rewrite fiber with different arithmetic costs;
- checkpoint schedules realize one derivative computation with different live-state traces.

9.5 The non-exact transport problem

A future projective rewrite geometry would need:

1. a category or 2-category of histories and semantics-preserving rewrites;
2. an edge transport not forced to be an endpoint coboundary;
3. a gauge transformation law and a class of coherence 2-cells;
4. at least one closed loop with a gauge-invariant nontrivial residue;
5. a comparison with the affine torsion of Paper 0 on the Borel sector.

Until these conditions are met, rewrite distance and filling are legitimate geometric quantities, while “projective rewrite curvature” remains an open programme.

10 Horner histories and ACS condensation

The first calibration stays inside the affine sector of AEG. It shows exactly how a commutative charge shadow can forget positional information while the evaluated operators remain distinct.

10.1 Radix histories

Fix an integer radix $k \geq 2$ and the digit alphabet $D_k = \{0, 1, \dots, k-1\}$. For $d \in D_k$, let

$$h_d(x) = kx + d.$$

A chronological word $w = d_1 d_2 \dots d_n$ acts by

$$\rho(w) = h_{d_n} \circ \dots \circ h_{d_1}.$$

Proposition 10.1 (Horner evaluation formula). *For every length- n word,*

$$\rho(w)(x) = k^n x + \sum_{i=1}^n d_i k^{n-i}. \quad (20)$$

Proof. Induction on n . Appending d_{n+1} sends the current value y to $ky + d_{n+1}$, multiplying all previous coefficients by k and adding the new least significant digit. $\square$

Theorem 10.2 (Radix-word injectivity). *Over $\mathbb{Z}$, or over a characteristic-zero field containing the displayed integers, distinct words in D_k^n induce distinct affine operators.*

Proof. All operators have slope k^n . Equality would force equality of the constants in Equation (20). These constants are the integers $0, 1, \dots, k^n - 1$ written uniquely in length- n base- k notation, allowing leading zeros. $\square$

The characteristic-zero hypothesis is active. Over a finite field, distinct integer constants may coincide after reduction; Section 12 uses such modular identification deliberately.

10.2 Condensation by ACS charge

Realize h_d as a scale by k followed by an addition of d . The ACS of Paper 0 [1] records only total additive and logarithmic multiplicative charge. Hence

$$\chi(w) = \left(\sum_{i=1}^n d_i, n \log k \right). \quad (21)$$

Theorem 10.3 (Fixed-histogram operator count). *Let m_r be the number of occurrences of digit r , with $\sum_{r=0}^{k-1} m_r = n$. All words with this histogram have the same ACS charge, and they induce exactly*

$$\frac{n!}{\prod_{r=0}^{k-1} m_r!} \quad (22)$$

distinct affine operators.

Proof. Equation (21) depends only on the histogram. The number of words with the histogram is the multinomial coefficient. They give distinct operators by Theorem 10.2. $\square$

Corollary 10.4 (Binary fixed-weight fiber). *For $k = 2$, the length- n words with exactly m ones have one ACS charge and $\binom{n}{m}$ distinct operator images. Under the uniform distribution on that set, conditioning only on the charge loses*

$$\log_2 \binom{n}{m}$$

bits of word information.

For $n = 4$ and $m = 2$, the six constants are

$$3, 5, 6, 9, 10, 12,$$

corresponding to the six placements of two ones. The charge remembers their number but not their positions.

10.3 Which state is sufficient?

For all future affine continuations, the current affine operator is sufficient: composition depends only on its slope and translation. The ACS charge is not sufficient, because two words with one charge can already have different translations before any future is applied. If the observation is deliberately coarsened to ACS charge and all future updates act only on charge, then the charge becomes an exact residual for that smaller experiment.

This illustrates the hierarchy

$$\text{word} \longrightarrow \text{affine operator} \longrightarrow \text{ACS charge}.$$

The size in Theorem 10.3 is the operator image of a charge fiber. It is not a fiber of operator evaluation.

10.4 Streaming cost and encoding cost

The recurrence

$$a_i = ka_{i-1} + d_i$$

evaluates a word in n affine steps while storing one current field element. In a unit-cost field-operation model this is $O(n)$ work and $O(1)$ field elements of workspace. Over the integers, however, the bit length of a_i is $\Theta(i \log k)$ in the worst case. Thus “one accumulator” does not mean constant bit space in a bit-complexity model.

The example therefore supplies an exact condensation count but no generic search or hardness theorem. Positional information can be large while direct streaming evaluation remains simple.

11 Cut order and residual width in decision diagrams

Ordered binary decision diagrams give a finite model in which contextual residual classes are represented directly as graph nodes. The example also shows that exponential cut sensitivity does not require noncommuting semantic operations.

11.1 Residual functions after a variable prefix

Let $F : \{0, 1\}^N \rightarrow \{0, 1\}$ and fix a variable order π . After assigning the first t variables according to a prefix α , the remaining behavior is the restricted function $F|_\alpha$.

Proposition 11.1 (Residual-width lower bound). *Every deterministic OBDD respecting π has, at the cut after the first t variables, at least as many distinct reachable nodes as there are distinct restricted functions $F|_\alpha$.*

Proof. If two prefixes reach the same node, every common continuation follows the same remaining subgraph and therefore gives the same Boolean answer. Hence the two restricted functions are equal. Distinct residual functions must reach distinct nodes. $\square$

For reduced OBDDs the converse also holds: isomorphic remaining subfunctions are merged. This is the decision-diagram instance of Theorem 5.5; see [3, 10].

11.2 Equality under block order

Define

$$\text{EQ}_n(x, y) = 1 \iff x = y, \quad x, y \in \{0, 1\}^n.$$

Consider the block order

$$x_1, \dots, x_n, y_1, \dots, y_n.$$

After all x -variables have been assigned to a string a , the residual function is

$$g_a(y) = \mathbf{1}[y = a].$$

Theorem 11.2 (Exponential middle width). *Every OBDD for EQ_n in block order has width at least 2^n at the middle cut.*

Proof. The functions g_a , $a \in \{0, 1\}^n$, are pairwise distinct. Apply Proposition 11.1. $\square$

The logarithm of this width is n bits, exactly the information needed to remember the first block for arbitrary comparison with the second.

11.3 Equality under interleaved order

Now use

$$x_1, y_1, x_2, y_2, \dots, x_n, y_n.$$

While equality has not failed, reading x_i creates one of two pending states, “the next y_i must be 0” or “the next y_i must be 1”. Reading y_i returns to the matching state or goes permanently to the zero terminal.

Theorem 11.3 (Constant interleaved width). *There is an OBDD for EQ_n in interleaved order with maximum width at most three and total size $O(n)$.*

Proof. Use the matching state, the two pending-bit states, and the shared zero terminal as described above. At any variable level at most three nonterminal or active residual states are needed, and each pair of variables contributes only a constant number of nodes. $\square$

The two orders therefore expose exponentially different residual widths for the same Boolean function and representation language.

11.4 The restrictions commute

Assignments to two distinct variables commute as semantic restriction operators:

$$(F|_{x_i=a})|_{x_j=b} = (F|_{x_j=b})|_{x_i=a}.$$

Nevertheless, the order in which variables cross the cut changes which partial information must remain available. The exponential separation in Theorems 11.2 and 11.3 is therefore a schedule and representation phenomenon, not a consequence of operator noncommutativity.

11.5 What the width theorem measures

At one cut, $\log_2(\text{width})$ lower-bounds the number of bits needed to select a residual node. The entire OBDD also stores transitions for all input prefixes, so its static node count is a different resource. Moreover, a program outside the OBDD model might compute equality with a different state organization. The theorem is exact for fixed variable order and the OBDD representation class; it is not a model-independent lower bound for every algorithm computing equality.

12 Butterflies, transform representations, and NTT states

Fast transforms illustrate representation change rather than endpoint shortcuts. A local butterfly has a precise projective quotient description, but a full transform is a multi-wire linear network and must retain data that one-wire projective semantics discards.

12.1 The butterfly torus-coset lemma

Let K be a field with $\text{char } K \neq 2$ and let $\omega \in K^\times$. Define

$$B_\omega = \begin{pmatrix} 1 & \omega \\ 1 & -\omega \end{pmatrix}, \quad B_1 = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad D_\omega = \begin{pmatrix} 1 & 0 \\ 0 & \omega \end{pmatrix}.$$

Proposition 12.1 (Butterfly torus coset). *In $\text{GL}_2(K)$,*

$$B_\omega = B_1 D_\omega. \tag{23}$$

After projectivization, if H is the diagonal stabilizer of $(0, \infty)$, then

$$B_\omega H = B_1 H.$$

Thus all nonzero twiddle butterflies have one quotient skeleton and a lift-dependent diagonal label.

Proof. Equation (23) is direct multiplication, and $D_\omega \in H$ after projectivization. The determinant -2ω is nonzero under the hypotheses. $\square$

This is a right- H -coset statement. It does not identify the two-wire gate with a one-hole arithmetic context, nor does it make the diagonal coordinate a canonical gauge-independent residue.

12.2 Why linear-group data remain necessary

The inverse is

$$B_\omega^{-1} = \frac{1}{2} \begin{pmatrix} 1 & 1 \\ \omega^{-1} & -\omega^{-1} \end{pmatrix}.$$

Projectivization forgets the common scalar, but exact inverse transforms and integer or finite-field arithmetic must retain normalization factors. A full FFT or NTT therefore belongs naturally to a linear multi-wire category with objects K^N , not solely to $\text{PGL}_2(K)$.

Construction 12.2 (Network AEG interface). A prospective multi-wire extension would have wire arities as objects, linear or arithmetic gates as morphisms, tensor product as parallel composition, and ordinary composition as layer gluing. It should contain the one-wire marked histories as a subcategory and admit a semantics functor to maps $K^m \rightarrow K^n$.

This construction is a structural proposal. Proposition 12.1 is the proved local interface.

12.3 Transform factorization of convolution

For a field containing a primitive N -th root of unity and with N invertible, the discrete Fourier transform F_N converts cyclic convolution into pointwise multiplication:

$$x * y = F_N^{-1}(F_N x \odot F_N y). \tag{24}$$

Radix factorizations reduce the transform from a dense matrix-vector product to $O(N \log N)$ local butterflies in the standard algebraic operation model; the classical radix-two algorithm is due to Cooley and Tukey [4].

For $N = 2^m$, a fixed breadth-first radix-two network has

$$D_{\text{net}} = m, \quad W_{\text{cell}} = \frac{N}{2}m \quad (25)$$

synchronous layers and butterfly cells, respectively.

Integer multiplication requires more than Equation (24): coefficient encoding, padding or a noncyclic construction, carry recovery, sufficiently large moduli or CRT, and bit-complexity analysis must also be charged.

12.4 A finite NTT calibration

Take $K = \mathbb{F}_{257}$. Fermat's theorem gives $3^{256} = 1$, while direct repeated squaring gives

$$3^{128} = -1 \pmod{257}.$$

Hence 3 has order 256. For $N \mid 256$ put

$$\zeta_N = 3^{256/N};$$

then ζ_N is a primitive N -th root. In particular, $\zeta_8 = 64$ and $\zeta_{16} = 249$.

Proposition 12.3 (Layer-state count). *Let a fixed radix-two NTT layer be invertible on $\mathbb{F}_{257}^N$ and let the input range be all of $\mathbb{F}_{257}^N$. Then the reachable state set at that layer has exactly 257^N elements. Its logarithmic cardinality is $N \log_2 257$ bits, while the direct fixed-width encoding by residues $0, \dots, 256$ uses $9N$ bits.*

Proof. An invertible layer is a bijection of $\mathbb{F}_{257}^N$, so its image is the whole space. Cardinality and the two encodings follow immediately. $\square$

For $N = 8$ the network has three layers and twelve butterflies; for $N = 16$ it has four layers and thirty-two butterflies.

12.5 Static labels versus dynamic states

A fixed transform's twiddle factors are program constants. They may be stored in a table, represented by a primitive root and exponent schedule, or generated on demand. Their storage-reconstruction tradeoff concerns L_{desc} , work, and perhaps cache traffic. It is not dynamic input entropy. Conversely, the N wire values vary with the input and belong to the live state.

The safe representation principle is therefore:

Seek a declared change of representation in which the core operation is diagonal, block-diagonal, sparse, or low-bandwidth, while charging the forward transform, inverse transform, static constants, numerical precision, and communication.

This principle motivates AEG-native representation search, but it is not a definition of every fast algorithm and does not prove an asymptotic improvement by itself.

13 Equal semantics with different work and space

The preceding examples varied semantic quotients and cut order. We now keep the final map fixed and vary its realization. Matrix parenthesization changes arithmetic work and intermediate shape; reverse-mode checkpointing exchanges stored history for recomputation.

13.1 Matrix-chain parenthesization

Let

$$A \in K^{d_0 \times d_1}, \quad B \in K^{d_1 \times d_2}, \quad C \in K^{d_2 \times d_3}.$$

Both $(AB)C$ and $A(BC)$ evaluate to the same linear map whenever the products are defined.

Proposition 13.1 (Three-matrix work and principal intermediates). *Under ordinary dense multiplication, counting scalar multiplications,*

$$W_L = d_0 d_1 d_2 + d_0 d_2 d_3, \quad R_L = d_0 d_2,$$

$$W_R = d_1 d_2 d_3 + d_0 d_1 d_3, \quad R_R = d_1 d_3,$$

where R_L and R_R are the numbers of scalar entries in the principal intermediate matrices AB and BC .

Proof. Multiplying an $r \times s$ matrix by an $s \times t$ matrix uses rst scalar multiplications in the stated algorithm and produces an $r \times t$ matrix. Apply this twice in each order. $\square$

For $(d_0, d_1, d_2, d_3) = (100, 10, 1, 2)$,

$$(W_L, R_L) = (1200, 100), \quad (W_R, R_R) = (2020, 20).$$

The lower-work plan has the larger principal intermediate. The R values are not complete peak-memory formulas: input retention, output allocation, overwrite policy, blocking, and temporaries still need to be declared.

For four matrices, the five parenthesizations form the vertices of an associahedral pentagon. If an edge is labelled by the difference of endpoint costs, Proposition 9.2 makes its sum around the pentagon zero. Positive work spent traversing the pentagon is action, not holonomy.

13.2 A chain and its reverse sweep

Consider a length- N composition

$$x_{i+1} = f_i(x_i), \quad 0 \leq i < N,$$

and its reverse accumulation

$$\bar{x}_i = Df_i(x_i)^* \bar{x}_{i+1}.$$

The reverse step requires the corresponding forward state x_i .

Assume each forward evaluation and each reverse local step has unit cost and each state has fixed size.

Proposition 13.2 (Two exact endpoint schedules). *The following schedules compute the same reverse result.*

1. Store all forward states. The total local work is $2N$ and the number of stored forward states is $\Theta(N)$.
2. Store only the input and recompute each required prefix. The total local work is

$$2N + \frac{N(N-1)}{2}, \quad (26)$$

and only $\Theta(1)$ states need be live, apart from the declared input, output, and control records.

Proof. The store-all schedule performs N forward and N reverse local operations. For the second schedule, the initial forward pass still costs N , the reverse steps cost N , and reconstructing x_i from x_0 costs i forward steps. Summing $i = 0, \dots, N-1$ gives Equation (26). $\square$

Checkpointing interpolates between these extremes. Under a specified number of checkpoints and a uniform linear chain, schedules such as REVOLVE optimize the recomputation pattern for their declared model and objective [5].

13.3 History condensation and re-expansion

A checkpoint is a materialized summary of the forward history sufficient to restart from that point. Erasing it condenses the live trace further; recomputation re-expands the missing segment from an earlier checkpoint. This interpretation is structurally compatible with AEG’s process-before-result principle, but it does not identify a nonlinear derivative pullback with one global projective inverse.

The example validates the live-configuration correction of Theorem 7.3. Both schedules eventually compute the same forward nodes and the same reverse result, yet their live states and work differ. No monotone set of ever-computed nodes can encode that tradeoff.

13.4 Reversible computation boundary

A logically reversible realization must retain or reconstruct information that a many-to-one step would otherwise erase. General reversible simulations therefore exhibit time–space tradeoffs; Bennett’s pebbling construction is a classical model [2]. The present paper uses this as an external calibration, not as a theorem that every AEG condensation has the same optimal tradeoff.

13.5 What equal semantics permits us to conclude

Matrix chains and checkpoint schedules prove that one semantic map can have incomparable resource vectors. They support the Pareto viewpoint of Equation (17). They do not show that all time and space costs are coordinates of one scalar “representation volume.” A unification would require a model-specific theorem relating semantic residuals, live encodings, and allowed re-expansions.

14 Claim boundaries and the remaining programme

The preceding sections prove a collection of comparison theorems, not a universal complexity law. This section gathers the claim boundary in one place and formulates the remaining AEG programme in falsifiable terms.

14.1 The four layers

The completed framework distinguishes:

$$\boxed{\begin{array}{c} \text{projective quotient fibers} \\ \text{Hist}_K \rightarrow G \rightarrow G/H \rightarrow G/B_{\pm} \end{array}} \quad \boxed{\begin{array}{c} \text{contextual residuals} \\ (P_C, M_C, O_C) \mapsto \mathcal{R}_C \end{array}} \quad (27)$$

$$\boxed{\begin{array}{c} \text{operational live configurations} \\ (\mathcal{M}, \Gamma, \sigma) \mapsto \mathbf{C}_{\mathcal{M}} \end{array}} \quad \boxed{\begin{array}{c} \text{rewrite fibers} \\ \rho^{-1}(y), \mathcal{R} \mapsto d_{\mathcal{R}}, \text{Fill}_{\mathcal{R}} \end{array}}. \quad (28)$$

The projective fiber is algebraic. The contextual residual is semantic and future-relative. The live configuration is machine-dependent and dynamic. The rewrite fiber organizes equal semantic realizations. The paper proves bridges only when the required actions and observations have been supplied.

14.2 Principal proved statements

Layer	Proved statement	Active boundary
projective	Regular bivaluations are rank-one idempotents and form G/H ; projective frames form an H -torsor.	No canonical connection or semantic interpretation of a projector is inferred.
continuation	Left futures descend to G/H ; one right update descends iff $k^{-1}Hk \subseteq H$; reversible right action requires the normalizer.	The conclusion depends on chronology and the selected quotient observation.
residual	$\mathcal{R}_C$ is the canonical minimal exact deterministic state; finite residuals give a fixed-width bit lower bound.	Randomized, approximate, infinite, and lossy models require new definitions.
ported programs	Strict finite substitution preserves sources and occurrences and commutes with evaluation on the exact composite domain; compatible collars and interfaces descend.	Global DAG, complete seam records, strict partial domains; no AES gate-motion realization is inferred.
features	Observation factorization holds iff feature fibers refine contextual equivalence; online updates hold iff the feature kernel is a right congruence.	Observation factorization, effectiveness, and actual opposite-edge witnesses are separate obligations.
operational	Live configurations, not ever-computed sets, determine workspace when erase or recompute actions exist.	Actual resources remain model-, encoding-, and schedule-dependent.
growth	Fiber-image and conditional-entropy inequalities are exact.	Word growth and fiber growth are not running-time lower bounds.
rewrite	Potential differences and pure-gauge labels telescope on closed loops.	Nontrivial holonomy needs non-exact transport and a gauge law.

Layer	Proved statement	Active boundary
Horner	A fixed digit histogram has a multinomial number of distinct operators with one ACS charge in characteristic zero.	The count is not a hardness result and can collide in finite characteristic.
OBDD	Equality has exponential block-order width and constant interleaved-order width.	This is a representation- and order-specific theorem.
transform	Butterflies occupy one projective torus coset; radix networks have exact layer and cell counts.	A full FFT/NTT is multi-wire GL data with scalar, precision, and communication costs.
time–space	Matrix chains and checkpoint schedules give equal semantics with incomparable resource vectors.	No machine-independent scalar unifies the coordinates.

The detailed hypothesis ledger appears in [Appendix D](#).

14.3 Excluded implications

The following implications are neither proved nor used:

noncommutativity $\implies$ negative curvature, negative curvature $\implies$ algorithmic hardness,

exponential group growth $\implies$ exponential evaluation time,

large history fiber $\implies$ large shared representation, endpoint cost difference $\implies$ holonomy.

Counter-calibrations occur inside the paper: free reduction is linear despite exponential free-group growth; OBDD order can create exponential width although variable restrictions commute; exact edge potentials telescope; and one streaming Horner accumulator evaluates a charge fiber containing many distinct operators.

14.4 Interfaces with Papers 0–III

Paper I supplies marked spinal histories and bilateral projective evaluation [11]; Paper 0 supplies the affine/Borel sector, ACS charge, torsion, and contact curvature [1]. Paper IV imports those objects but does not rewrite their foundational proofs. The quotient tower and residual theory begin after operator evaluation.

Paper II supplies analytic representations and finite affine–Appell families [12]. They motivate the search for coordinates in which selected operators become sparse or triangular. Paper IV does not claim function-space completeness or a new asymptotically fast transform from those families.

Paper III supplies singular zero networks, arithmetic register fibers, monodromy, and explicit forgetting phenomena [13]. Those examples demonstrate that finite forgetting maps can lose geometric information. Paper IV does not identify the Lyashko–Looijenga forgetting variance, braid monodromy, or tube energy with computational complexity.

The dependency remains one-way:

Papers I–III provide typed examples $\longrightarrow$ Paper IV supplies quotient and resource comparisons.

14.5 Open problems

Open problem 14.1 (Projective torsion). Construct a gauge-equivariant comparison on $G \rightarrow G/H$ that reduces to affine translation torsion in the Borel sector and has a nontrivial closed-loop observable not forced to telescope.

Open problem 14.2 (Projective-to-contextual bridge). Classify continuation experiments for which an H -lift, a quotient point gH , or a conjugacy class of relative defects is a complete contextual residual. Determine sharp refinements when right and left futures are both allowed.

Open problem 14.3 (Faithful multi-wire AES realization). Extend the finite ported-network interface of Section 6 to a symmetric monoidal category or PROP with a faithful AES state realization. Supply a multi-hole configuration or fiber carrier and realize the actual typed arithmetic and structural generators, including binary gates, copy, discard, and their ordinary domains. Retain sources, occurrences, literal call receipts, and full-state residuals. Prove execution transport, overlap conjugacy, and independence of decomposition. Recover the marked-history one-wire fragment and a linear sector containing butterfly networks, retaining central scalars. The finite attached-network theorem settles neither these geometric realizations nor these embedding requirements.

Open problem 14.4 (Opposite-edge feature witnesses). Construct the actual three characteristic witnesses in a declared Adva experiment [16]. Determine whether they satisfy Theorem 6.9 and Proposition 6.10, or which retained process data are needed to refine them. Declare operator closure before using spectra and snapshot/update or fixed-point semantics before using three-side feedback. The scalar outputs in Example 6.12 do not identify these native output roles.

Open problem 14.5 (Costed representation change). For a declared transform class Φ , define and analyze

$$\mathcal{C}(F; \Phi) = \mathcal{C}(\Phi) + \mathcal{C}(\Phi F \Phi^{-1}) + \mathcal{C}(\Phi^{-1})$$

with precision, conditioning, static constants, communication, and uniformity included. Exhibit an AEG-native family for which the optimized representation provably improves a baseline model.

Open problem 14.6 (Approximate residuals). Replace exact equality in Definition 5.1 by a metric, probabilistic, or decision-theoretic tolerance. Determine which covering, packing, mutual-information, or rate-distortion quantities replace $\log_2 |\mathcal{R}_C|$.

Open problem 14.7 (History-natural cost functor). Construct a functor from a typed history or network category to a resource category that records legal simulation, composition, erasure, and recomputation. Prove which resource coordinates are subadditive, additive, or monotone under composition.

Open problem 14.8 (Model robustness). Identify a class of machine models connected by efficient simulations for which the Pareto geometry in Equation (17) is stable up to declared distortions.

Open problem 14.9 (Rosetta graph families). Use chains, trees, diamonds, grids, and butterflies as common graph topologies for expression evaluation, proof search, tensor contraction, differentiation, and transforms. Determine which residual and resource quantities are graph invariants and which depend essentially on values, futures, or representations.

These problems are deliberately separated. Solving one does not automatically solve the others.

15 Conclusion

Arithmetic Expression Geometry begins before endpoint evaluation. A marked history may be replaced by a projective operator, a rank-one projector, a single projective point, or a numerical

value. Paper IV has treated the fibers of these replacements as mathematical objects while refusing to identify fiber size with computational cost without an intervening model.

The projective algebra is exact. Regular bivaluations are rank-one idempotents, G/H is their homogeneous space, and $G \rightarrow G/H$ is the frame torsor. Over a finite field every cardinality is explicit. More importantly, the interaction with continuation can be decided: left composition acts on G/H , while a right update descends only under the conjugation inclusion $k^{-1}Hk \subseteq H$, and a reversible right action lies in the normalizer. The same quotient can therefore be complete for one chronology and incomplete for another.

Contextual residual equivalence turns this observation into a general exact state theorem. Once a typed interface, legal future monoid, partial-domain convention, and observation are fixed, the residual quotient is the canonical minimal deterministic online state. Finite residual counts give fixed-width state-selection bounds. Operational resource claims require more: a live configuration trace, encoding, actions, schedule, and charging rule. This is why checkpoint erasure and recomputation cannot be represented by a monotone set of nodes ever computed.

The ported-AES interface makes finite program substitution explicit without changing the geometric assignment at its ends. Strict finite DAGs retain sources, fresh occurrences, explicit copy/discard, and exact partial domains. They compose associatively at the numerical level and up to an explicit isomorphism at the network level, while call receipts remain distinct. Compatible collar and seam data descend under global acyclicity. Features support all future observations exactly when their fibers refine contextual equivalence; online updates require the additional right-congruence condition. These results concern attached programs, not yet faithful AES gate motions.

The case studies show several distinct ways in which order matters. Horner order changes the affine operator while ACS charge stays fixed. OBDD variable order changes exposed residual width although restrictions commute. Transform factorization changes representation and communication. Matrix parenthesization and checkpointing change work and live space while semantics stay fixed. Rewrite potential differences, by contrast, are exact and cannot produce nontrivial loop holonomy.

The resulting architecture is not a claim that representation complexity, time, and space are already one invariant. It is a disciplined interface for asking when they can be compared:

$$\text{history} \longrightarrow \text{quotient} \longrightarrow \text{future residual} \longrightarrow \text{operational realization}.$$

Each arrow requires explicit data, and each comparison theorem states those data. The next stage is constructive: non-flat projective transport, faithful multi-wire AES motion realizations, actual opposite-edge features, costed representation changes, approximate residuals, and robust simulations across machine models.

A Projective and homogeneous-space calculations

This appendix fixes matrix, action, and chart conventions used in the projective chapters.

A.1 One-hole arithmetic contexts

Matrices act on the standard affine coordinate z by

$$\begin{pmatrix} \alpha & \beta \\ \gamma & \delta \end{pmatrix} \cdot z = \frac{\alpha z + \beta}{\gamma z + \delta}.$$

For $c \neq 0$ when required, the elementary contexts have representatives

<i>context</i>	<i>map</i>	<i>matrix</i>
$z + c$	$z \mapsto z + c$	$\begin{pmatrix} 1 & c \\ 0 & 1 \end{pmatrix}$
$z - c$	$z \mapsto z - c$	$\begin{pmatrix} 1 & -c \\ 0 & 1 \end{pmatrix}$
$c - z$	$z \mapsto c - z$	$\begin{pmatrix} -1 & c \\ 0 & 1 \end{pmatrix}$
cz	$z \mapsto cz$	$\begin{pmatrix} c & 0 \\ 0 & 1 \end{pmatrix}$
z/c	$z \mapsto z/c$	$\begin{pmatrix} 1 & 0 \\ 0 & c \end{pmatrix}$
c/z	$z \mapsto c/z$	$\begin{pmatrix} 0 & c \\ 1 & 0 \end{pmatrix}$

Ordinary arithmetic retains the domain restriction for c/z at $z = 0$; projective evaluation sends 0 to ∞ . A zero multiplier or a constant map belongs to a projective matrix monoid rather than $\text{PGL}_2(K)$.

A.2 Generation of the projective group

Let

$$T_b(z) = z + b, \quad D_a(z) = az, \quad J(z) = -1/z.$$

Theorem A.1 (Bilateral generators). *Translations T_b , nonzero scalings D_a , and J generate $\text{PGL}_2(K)$.*

Proof. Let

$$f(z) = \frac{Az + B}{Cz + D}, \quad AD - BC \neq 0.$$

If $C = 0$, then f is a nonzero scaling followed by a translation. If $C \neq 0$, direct substitution gives

$$f = T_{A/C} \circ D_{(AD-BC)/C^2} \circ J \circ T_{D/C}. \quad (29)$$

Indeed the right side is

$$\frac{A}{C} - \frac{AD - BC}{C^2(z + D/C)} = \frac{Az + B}{Cz + D}.$$

□

The contexts that fix ∞ form the affine Borel

$$B_\infty = \text{Stab}_G(\infty) \cong \text{Aff}(1, K).$$

The second-slot division c/z is the elementary arithmetic context that leaves this Borel and enters the other Bruhat cell.

A.3 Point–predicate covariance

For $g \in \text{GL}(V)$, point and copoint representatives transform by

$$v \mapsto gv, \quad \varphi \mapsto \varphi g^{-1}.$$

Hence the pairing is invariant:

$$(\varphi g^{-1})(gv) = \varphi(v),$$

and the rank-one projector is conjugated:

$$\Pi_{gv, \varphi g^{-1}} = g \Pi_{v, \varphi} g^{-1}. \quad (30)$$

This explains why a bivaluation is an image–kernel result rather than two independent scalar coordinates.

A.4 Local affine bivaluation law

Use the local representatives of Equation (6). Suppose the point coordinate obeys

$$a \mapsto Aa + B, \quad A \neq 0.$$

In the standard coordinate $z = -a$, a representative is

$$G_{A,B} = \begin{pmatrix} A & -B \\ 0 & 1 \end{pmatrix}.$$

Let $p = -1/b$ be the projective point whose line is the kernel of φ_b . Since p transforms as a point,

$$p \mapsto Ap + B.$$

Solving again for b gives

$$b \mapsto \frac{b}{A - Bb}. \quad (31)$$

A zero denominator in Equation (31) can be a chart escape even when the global ordered pair remains transverse. Incidence failure is instead $1 + ab = 0$.

For infinitesimal positive-real affine parameters $A = e^{\lambda dv}$ and $B = \mu du$, the first-order laws are

$$da = \mu du + \lambda a dv, \quad db = \mu b^2 du - \lambda b dv, \quad dp = \mu du + \lambda p dv.$$

These are a point/dual-point transport pair. They are not the same binary axis as operand-slot chirality.

A.5 Frame coordinate above a bivaluation

Choose the reference ordered pair $(0, \infty)$. A third point is then any $t \in K^\times$. The stabilizer

$$H = \{z \mapsto az : a \in K^\times\}$$

acts by $t \mapsto at$, freely and transitively. Thus the frame fiber is a copy of $K^\times$ only after this reference pair and affine coordinate have been chosen. Under a different local section, the displayed H -coordinate changes by the usual gauge multiplication.

A.6 Direct normalizer calculation

Assume $|K| > 2$ and let a projective matrix g normalize the diagonal torus. The common fixed set of the torus is $\{0, \infty\}$, so g preserves this set. If it fixes both points, it has a diagonal representative

$$\begin{pmatrix} a & 0 \\ 0 & d \end{pmatrix};$$

if it interchanges them, it has an antidiagonal representative

$$\begin{pmatrix} 0 & b \\ c & 0 \end{pmatrix}.$$

Modulo scalars these are respectively H and JH . This is the matrix form of Proposition 4.6.

A.7 Finite-field count from ordered frames

An ordered projective frame over $\mathbb{F}_q$ is a triple of pairwise distinct points. There are

$$(q+1)q(q-1) = q(q^2-1)$$

such triples, exactly $|\mathrm{PGL}_2(\mathbb{F}_q)|$. Forgetting the third point leaves $(q+1)q$ bivaluations and a fiber of $q-1$ possible frame points. This gives a second proof of Proposition 3.7 without counting invertible matrices.

B Residual proofs and partial-domain conventions

This appendix expands the residual construction, especially the partial-domain and encoding conventions suppressed in the main flow.

B.1 Why an invalid outcome is necessary

Suppose a continuation class contains the ordinary future $z \mapsto 1/z$ but one compares only outputs on futures legal for both pasts. The current value 0 has no legal output for this future. A common-domain convention can then make its comparison vacuous. Totalization instead records

$$O(0 \cdot (1/z)) = \perp, \quad O(1 \cdot (1/z)) = 1,$$

so the two pasts are correctly distinguished. The same device handles type errors, missing registers, and forbidden resource actions.

Associativity remains valid after totalization: once a computation enters $\perp$, every further continuation stays there. Therefore the proof of the right-congruence proposition requires no special cases.

B.2 The factorization diagram

For an exact deterministic machine, Theorem 5.5 produces the commutative diagram

$$\begin{array}{ccc} P_C & \xrightarrow{e} & e(P_C) \\ & \searrow p \mapsto [p] & \downarrow \phi \\ & & \mathcal{R}_C \end{array}$$

Every transition commutes with ϕ , and the output factors through $\mathcal{R}_C$. Thus a larger exact state representation may contain caches or redundancy, but it cannot merge two residual classes.

B.3 One-way communication interpretation

At a cut, let Alice know the past p and Bob choose the future η . Alice sends one deterministic message $m(p)$; Bob must output $O_C(p \cdot \eta)$ for every legal pair.

Proposition B.1 (Deterministic one-way lower bound). *Any such exact protocol uses distinct messages for distinct residual classes. A fixed-width worst-case message therefore has length at least $\lceil \log_2 |\mathcal{R}_C| \rceil$.*

Proof. If $m(p) = m(p')$, Bob behaves identically for every chosen future. Exactness then gives $p \equiv_C p'$. The counting bound follows. $\square$

This interpretation makes clear why the future class matters: it is Bob's set of allowed tests.

B.4 Dense tensor boundary example

Let the state exposed at a cut be a dense tensor represented as a vector $T \in \mathbb{F}_q^D$. If every linear functional $\ell \in (\mathbb{F}_q^D)^\vee$ is an allowed future observation, then two states are residually equivalent exactly when they are equal. Indeed, if $T - T' \neq 0$, a coordinate functional detects a nonzero component. Hence

$$|\mathcal{R}_C| = q^D, \quad \log_2 |\mathcal{R}_C| = D \log_2 q.$$

A direct fixed-width residue encoding uses $D \lceil \log_2 q \rceil$ bits.

If only one nonzero functional ℓ is allowed, the residual quotient is instead $\mathbb{F}_q^D / \ker \ell$, which has q classes. The boundary dimension did not change; the legal future class did. Sparse, low-rank, symmetric, or approximate tensor models change the representation again.

B.5 Prefix-free variable-length states

Let N residual classes receive a prefix-free binary code with maximum length b . Kraft's inequality gives

$$1 \geq \sum_{i=1}^N 2^{-\ell_i} \geq N 2^{-b},$$

so $b \geq \lceil \log_2 N \rceil$. Without prefix-freeness or another decoding convention, a raw collection of variable-length strings cannot be counted as a self-delimiting state code.

Average code length obeys distribution-sensitive entropy bounds, while the main theorem is a worst-case state-selection statement. The two should not be interchanged.

B.6 Infinite residual spaces

When $\mathcal{R}_C$ is infinite, cardinal logarithms do not directly give finite bit bounds. One may need:

- a topology and covering or packing numbers at scale ε ;
- a measure and Shannon or mutual information;
- a metric loss and rate-distortion function;
- an algebraic dimension or rank;

- a computable presentation and description-length convention.

No one of these is canonical without an application-specific observation and approximation criterion.

B.7 Changing future classes

Let $M_1 \subseteq M_2$ be future classes with the same pasts and observation. Then

$$p \equiv_{M_2} p' \implies p \equiv_{M_1} p'.$$

Thus there is a natural surjection

$$P/\equiv_{M_2} \longrightarrow P/\equiv_{M_1}.$$

Adding tests refines the residual space; removing tests condenses it. The projective passage from left-only futures to arbitrary two-sided futures is a group-theoretic instance of this general refinement.

C Finite network and resource counts

The tables below make the finite calibrations reproducible without changing their interpretation.

C.1 The finite-field projective tower

$$|\mathbb{P}^1(\mathbb{F}_7)| = 8, \quad |\text{Biv}_{\mathbb{F}_7}^{\text{reg}}| = 8 \cdot 7 = 56, \quad |\text{PGL}_2(\mathbb{F}_7)| = 8 \cdot 7 \cdot 6 = 336.$$

Every bivaluation has six projective-frame lifts.

C.2 Binary Horner words of length four

For $k = 2$, $n = 4$, and Hamming weight two:

word $(d_1 d_2 d_3 d_4)$	constant in $16x + c$
0011	3
0101	5
0110	6
1001	9
1010	10
1100	12

All six words have charge $(2, 4 \log 2)$.

For $n = 8$, the fixed-weight counts are:

m	0	1	2	3	4	5	6	7	8
$\binom{8}{m}$	1	8	28	56	70	56	28	8	1

The central charge class therefore contains seventy distinct affine operators in characteristic zero.

C.3 Free-group ball count

For the free group F_r with the standard symmetric alphabet of size $2r$, the sphere of radius $n \geq 1$ has

$$2r(2r-1)^{n-1}$$

reduced words. Hence

$$|B(n)| = 1 + 2r \frac{(2r-1)^n - 1}{2r-2}.$$

The exponential metric growth coexists with the linear-time stack algorithm for freely reducing one input word.

C.4 Finite-field roots and transform networks

Repeated squaring gives

$$3^{128} \equiv 256 \equiv -1 \pmod{257}, \quad 3^{256} \equiv 1 \pmod{257}.$$

Thus 3 has order 256. The selected roots and network counts are:

N	$\zeta_N = 3^{256/N}$	layers	butterflies	$\log_2$ state count	direct bits
8	64	3	12	$8 \log_2 257$	72
16	249	4	32	$16 \log_2 257$	144

The “direct bits” column uses nine bits per field element. The logarithmic cardinality is smaller because 257 is not a power of two. Twiddle constants are not included in either dynamic-state column.

C.5 Matrix-chain calibration

For dimensions $(100, 10, 1, 2)$:

<i>plan</i>	scalar multiplications	principal intermediate entries
(AB)C	$100 \cdot 10 \cdot 1 + 100 \cdot 1 \cdot 2 = 1200$	100
A(BC)	$10 \cdot 1 \cdot 2 + 100 \cdot 10 \cdot 2 = 2020$	20

Peak memory cannot be recovered from the last column without an allocation and retention policy.

C.6 Reverse-chain endpoint schedules

Using Proposition 13.2:

N	store-all work	naive recompute work	extra prefix work
8	16	44	28
16	32	152	120

The work convention charges one unit per forward local evaluation and one per reverse local step. It excludes instruction, communication, and variable-size state costs.

D Claim ledger and cost-model checklist

This ledger records the status and boundary of the principal Paper IV claims. “Proved” means that a proof appears in the active manuscript under the stated hypotheses. “Framework” denotes a definition or proposed architecture, not a theorem of machine independence.

D.1 Claim ledger

ID	Claim	Status	Boundary
PIV-H0	Histories, operators, bivaluations, projective points, and endpoints form distinct equality levels.	Proved by definitions and examples	Ordinary-domain labels remain separate from projective continuation.
PIV-P1	Regular bivaluations are rank-one idempotents.	Proved, Theorem 3.2	Requires transverse ordered lines.
PIV-P2	$G/H \cong \text{Biv}_K^{\text{reg}}$ and $G \rightarrow G/H$ is a frame torsor.	Proved	No canonical connection over an arbitrary field.
PIV-P3	Finite-field quotient and fiber counts are exact.	Proved	Algebraic count, not dynamical attraction.
PIV-C1	A left-equivariant quotient with injective identity observation is the exact residual.	Proved	Depends on descended action and observation.
PIV-C2	One right update descends iff $k^{-1}Hk \subseteq H$.	Proved	Inclusion need not be equality for infinite H .
PIV-C3	Reversible right continuation acts iff it lies in $N_G(H)$.	Proved	Applies to right action on the selected right-coset quotient.
PIV-R1	Contextual residual equivalence is a right congruence.	Proved	Requires totalized partial action.
PIV-R2	The residual quotient is the canonical minimal exact deterministic state.	Proved	Exact deterministic online model only.
PIV-R3	Finite residuals lower-bound fixed-width or prefix-free maximum state length.	Proved	Not transition-table or average-code size.
PIV-R4	Summed residual bounds give fixed-register capacity–time.	Proved	Actual memory–time only under the declared snapshot charging rule.
PIV-S0	Ported AES carriers separate geometric ends, computational holes, and input/output ports.	Structural proposal	Port anchoring does not realize gates as AES motions.
PIV-S1	Typed finite substitution preserves sources and instantiated occurrences.	Proved with stated hypotheses	Complete one-use bindings; explicit copy/discard; global DAG.

ID	Claim	Status	Boundary
PIV-S2	Substitution commutes with strict partial evaluation on the exact composite domain.	Proved, Theorem 6.5	Pure deterministic primitives; discarded producers still execute.
PIV-S3	Composition is associative for partial maps and up to explicit isomorphism for operation networks.	Proved	Domains agree; literal receipts remain distinct; boundary orders must match.
PIV-S4	Compatible collars and computational interfaces descend together.	Proved with stated hypotheses	Smooth Hausdorff carrier, equal signed parameters, compatible full seam data and global DAG.
PIV-S5	Feature observations factor iff feature fibers refine contextual equivalence.	Proved, Theorem 6.9	Declared futures and invalid outcome; no effectiveness claim.
PIV-S6	Online feature updates exist iff the feature kernel is a right congruence.	Proved, Proposition 6.10	An exact machine also needs current-output factorization.
PIV-S7	The four-input scalar experiment outputs $(9, 15, 5/2)$.	Exact finite example	Attached network; not Adva's native output roles or an AES gate-motion realization.
PIV-O1	Ever-computed node sets do not determine workspace with erasure or recomputation.	Proved	Requires a positive-size optional live value.
PIV-G1	Maximum fiber size is at least raw words divided by image size.	Proved	Cardinality only.
PIV-G2	Conditional lost information obeys Equation (19).	Proved	Uniform word distribution in the displayed form.
PIV-W1	Additive potential labels telescope.	Proved	Single-valued additive potential.
PIV-W2	Pure-gauge group labels telescope.	Proved	Does not exclude non-flat connections.
PIV-H1	Fixed digit histograms give multinomially many distinct Horner operators with one ACS charge.	Proved	Characteristic zero; fixed radix and length.
PIV-B1	Equality OBDD width is exponential in block order and constant in interleaved order.	Proved	Fixed OBDD model and variable order.
PIV-N1	$B_\omega = B_1 D_\omega$ and all twiddles share one right H -coset.	Proved	Local 2×2 statement, char $K \neq 2$.
PIV-N2	Radix-two layer, cell, and invertible layer-state counts are exact.	Proved	Fixed breadth-first topology and full input range.

ID	Claim	Status	Boundary
PIV-T1	Matrix chains and reverse schedules realize exact work–space tradeoffs.	Proved in stated elementary models	Not a universal optimality theorem.
PIV-F1	Pareto resource geometry organizes uniform realizations.	Framework	Model, units, precision, and simulations must be fixed.
PIV-F2	Faithful multi-wire AES motion realizations and actual opposite-edge features.	Open problems	The finite ported-network interface is established; geometric gate realization and native feature witnesses are not.
PIV-F3	Projective rewrite holonomy should extend affine torsion.	Open problem	Exact frame and potential labels are known to telescope.
PIV-X1	Noncommutativity, curvature, group growth, and raw fibers imply hardness.	Excluded	No such implication appears in the active claims.

D.2 Cost-model checklist

Before any complexity comparison is promoted to a theorem, the manuscript or experiment must state:

1. the input family and size parameter;
2. the semantic task and exact, randomized, or approximate success rule;
3. the history, circuit, network, proof, or machine language;
4. uniformity and preprocessing;
5. ordinary-domain and type-error conventions;
6. value, instruction, constant, and output encodings;
7. atomic actions and their weights;
8. processor, memory, and communication topology;
9. live-state legality, erasure, and recomputation rules;
10. whether input, output, tables, code, and constants count toward space;
11. precision, conditioning, and numerical stability when applicable;
12. the continuation class and observation defining residuals;
13. the exact comparison or simulation relating the models.

A missing item does not make a calculation useless; it limits which resource claim the calculation can support.

D.3 Case-study matrix

Example	Semantic quotient	Operational object	Main lesson
ported AES	program substitution → strict partial evaluation	explicit-copy DAG and call receipts	attached program semantics do not realize AES gate motions
Horner	word → operator → ACS charge	streaming accumulator	large charge class can be easy to evaluate
OBDD equality	prefixes → residual Boolean functions	decision-diagram nodes	cut order can change width exponentially
NTT	dense transform → factored network; local G/H coset	wire layers and twiddle storage	static labels and dynamic state differ
matrix chain	parenthesizations of one product	intermediate matrices	equal semantics have incomparable resource vectors
reverse chain	one derivative map	checkpoints and recomputation trace	completed sets do not encode live space
rewrite loop	semantic fiber	edge actions and 2-cells	exact potentials are not holonomy

D.4 Release status

This manuscript is a mathematical-review integration. It promotes the proved finite and algebraic results listed above, while keeping machine-independent complexity, faithful multi-wire AES motion realizations, actual opposite-edge feature witnesses, projective holonomy, and approximate residuals open. It does not alter the release or DOI status of Papers I–III. Public release, author metadata, and DOI assignment remain subject to explicit author approval.

References

- [1] DeepSeek Harness Agent and Mingli Yuan. Arithmetic expression geometry 0—arithmetic expression spaces: Expansion, motion, ripple duals, commutativization, and tearing. Mathematical-review manuscript. First author is an artificial-intelligence research agent (`deepseek-v4-flash-vision-exp`); correspondence and responsibility rest with the second author, 2026.
- [2] Charles H. Bennett. Time/space trade-offs for reversible computation. *SIAM Journal on Computing*, 18(4):766–776, 1989.
- [3] Randal E. Bryant. Graph-based algorithms for boolean function manipulation. *IEEE Transactions on Computers*, C-35(8):677–691, 1986.
- [4] James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex fourier series. *Mathematics of Computation*, 19(90):297–301, 1965.
- [5] Andreas Griewank and Andrea Walther. Algorithm 799: REVOLVE: An implementation of checkpointing for the reverse or adjoint mode of computational differentiation. *ACM Transactions on Mathematical Software*, 26(1):19–45, 2000.

- [6] M. Gromov. Groups of polynomial growth and expanding maps. *Publications Mathématiques de l’IHÉS*, 53:53–78, 1981.
- [7] Anil Nerode. Linear automaton transformations. *Proceedings of the American Mathematical Society*, 9(4):541–544, 1958.
- [8] M. H. A. Newman. On theories with a combinatorial definition of “equivalence”. *Annals of Mathematics*, 43(2):223–243, 1942.
- [9] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3–4):379–423, 623–656, 1948.
- [10] Ingo Wegener. *Branching Programs and Binary Decision Diagrams: Theory and Applications*. Society for Industrial and Applied Mathematics, Philadelphia, 2000.
- [11] Mingli Yuan. Arithmetic expression geometry i: Foundations—sequential histories, projective semantics, and zero geometry. Mathematical-review manuscript, 2026.
- [12] Mingli Yuan. Arithmetic expression geometry ii: Hyperbolic real function theory— horizontal operators, boundary problems, and arithmetic holomorphicity. Mathematical-review manuscript, 2026.
- [13] Mingli Yuan. Arithmetic expression geometry iii: Singular zero geometry and tubes— branched pullbacks, arithmetic zero networks, and topological transport. Mathematical-review manuscript, 2026.
- [14] Mingli Yuan. Neutral carrier and triadic mechanism frontiers, 2026. Research note 0109, pinned at commit cce7300; outer arity and inner holes are distinct.
- [15] Mingli Yuan. Technical report: The program-process core of adva, 2026. Pinned research report at commit cce7300; finite program substitution and program/value distinction, Sections 1.2–1.3.
- [16] Mingli Yuan. Three-sided trace-arithmetic calibration, 2026. Research note 0114, pinned at commit cce7300; all three opposite-side characteristic witnesses remain open in the recorded version.